

ВКР

Петров А.Е.

Аннотация

Выпускная квалификационная работа посвящена реализации и верификации нейросетевой архитектуры BTNet для оценки американских пут-опционов. В основе рассматриваемого подхода лежит представление обратной индукции биномиальной модели Кокса-Росса-Рубинштейна в виде прямого прохода нейронной сети. Для европейского опциона используются сверточные слои, а для американского опциона - maxout-слои, реализующие выбор между продолжением позиции и досрочным исполнением. В работе был создан программный комплекс `bttn_bs` на Python с использованием PyTorch, реализованы модели BTNetEuropean и BTNetAmerican, проведена интеграция с QuantLib для эталонной проверки результатов, а также исследовано вычисление греческих символов Delta, Gamma, Vega и Theta средствами автоматического дифференцирования. Численные эксперименты показывают, что аналитическая инициализация весов на основе параметров CRR обеспечивает высокую точность оценки американского пут-опциона относительно QuantLib CRR; дополнительно показано, что перенос весов из европейской модели в американскую ухудшает точность цен и чувствительностей. Практическая значимость работы заключается в создании дифференцируемого инструмента для оценки опционов и анализа рыночного риска.

Abstract

The final qualifying work is devoted to the development and verification of the BTNet neural network architecture for pricing European and American put options. The study presents the analytical equivalence between BTNet and the Cox–Ross–Rubinstein binomial tree, implements BTNetEuropean and BTNetAmerican models in PyTorch, integrates QuantLib for pricing accuracy validation, and investigates the calculation of Greeks (Delta, Gamma, Vega, Theta) using automatic differentiation. An experiment on transferring weights from the European model to the American model is conducted to assess the impact on pricing and Greeks accuracy. The practical significance of the work lies in providing a software package for applying differentiable neural network models to option pricing and risk analysis in financial markets.

Оглавление

Список сокращений	4
Список основных обозначений	5
Введение	6
Глава 1. Теоретические основы	9
1.1. Опционы и задача оценки производных финансовых инструментов.....	9
1.2. Модели Блэка-Шоулса и Кокса-Росса-Рубинштейна.....	10
1.3. Нейросетевая эквивалентность BTNet.....	12
Глава 2. Реализация программного комплекса btnn_bs	26
2.1. Архитектура пакета и вычислительная среда	26
2.2. Реализация моделей BTNetEuropean и BTNetAmerican	28
2.3. Интеграция QuantLib и методика верификации	32
Глава 3. Численные эксперименты и вычисление греческих символов	36
3.1. Верификация точности прайсинга	36
3.2. Греческие символы через автоматическое дифференцирование	38
3.3. Эксперимент переноса весов из европейской модели в американскую	44
Заключение	48
Список используемых источников	51
Приложение А. Исходный код пакета btnn_bs	52
A.1. Базовые слои (layers.py).....	52
A.2. Архитектура BTNetEuropean (model_european.py)	54
A.3. Архитектура BTNetAmerican (model_american.py).....	55
A.4. Функция обучения train_BTNet (training.py)	57
A.5. Вычисление греческих символов через autograd (greeks.py)	58

Список сокращений

Русскоязычные сокращения

ВКР — выпускная квалификационная работа

ПК — программный комплекс

БШ — Блэк-Шоулз

Англоязычные сокращения

API — Application Programming Interface (интерфейс прикладного программирования)

BAW — Barone-Adesi-Whaley (аналитическое приближение для американских опционов)

BS — Black-Scholes (модель ценообразования опционов)

CRR — Cox-Ross-Rubinstein (биномиальная модель ценообразования)

MAE — Mean Absolute Error (средняя абсолютная ошибка)

RMSE — Root Mean Square Error (среднеквадратичная ошибка)

Список основных обозначений

S_0 — текущая цена базового актива

K — цена исполнения опциона (страйк)

T — время до экспирации, лет

r — безрисковая процентная ставка

σ — волатильность доходности базового актива

$V(S, t)$ — цена опциона при цене актива S в момент времени t

Δ (Delta) — производная цены опциона по цене базового актива, dV/dS_0

Γ (Gamma) — вторая производная цены по S_0 , d^2V/dS_0^2

v (Vega) — производная цены опциона по волатильности, $dV/d\sigma$

Θ (Theta) — производная цены опциона по времени, $-dV/dT$

u, d — коэффициенты роста и убывания цены в биномиальной модели CRR

p — риск-нейтральная вероятность роста цены актива

n — число временных шагов биномиального дерева

Δt — длина одного временного шага, $\Delta t = T/n$

W — весовой фильтр нейросетевой архитектуры BTNet

Введение

Производные финансовые инструменты занимают важное место в современной финансовой системе, поскольку используются для хеджирования рисков, спекулятивных стратегий и управления инвестиционными портфелями. Одним из наиболее распространенных классов деривативов являются опционы, стоимость которых зависит от цены базового актива, времени до экспирации, волатильности и процентной ставки. Для практического применения недостаточно определить только справедливую цену опциона: участникам рынка также необходимы чувствительности цены к рыночным параметрам, то есть греческие символы Delta, Gamma, Vega и Theta [1].

Особую сложность представляет оценка американских опционов, поскольку они допускают досрочное исполнение в любой момент до даты экспирации. Для американского пут-опциона право раннего исполнения имеет экономическую ценность и превращает задачу ценообразования в задачу оптимальной остановки со свободной границей. В отличие от европейского опциона, цена которого в классических предположениях может быть вычислена по формуле Блэка-Шоулса-Мертона [2; 3], для американского пут-опциона не существует универсального замкнутого аналитического решения [1]. Поэтому на практике применяются численные методы, в частности биномиальная модель Кокса-Росса-Рубинштейна [4].

Биномиальное дерево CRR позволяет последовательно вычислять цену опциона методом обратной индукции: сначала задаются выплаты в конечных узлах дерева, затем в каждом предыдущем узле вычисляется дисконтированная ожидаемая стоимость продолжения позиции. Для американского опциона на каждом шаге дополнительно сравниваются стоимость продолжения и стоимость немедленного исполнения [4]. Такой подход прозрачен с финансовой точки зрения, однако при многократном пересчете цен и чувствительностей может быть вычислительно затратен и не всегда удобен для прямого применения методов автоматического дифференцирования.

В настоящей работе рассматривается нейросетевая архитектура BTNet, основанная на теоретических результатах Шорохова С. Г. [5]. Ключевая идея BTNet состоит в том, что операции обратной индукции в биномиальном дереве могут быть представлены как прямой проход нейронной сети специальной структуры. Для

европейского пут-опциона используются полносвязный слой и последовательность одномерных сверточных слоев, а для американского пут-опциона сверточные слои заменяются maxout-слоями, реализующими выбор максимума между продолжением и досрочным исполнением. Такая конструкция сохраняет финансовую интерпретируемость CRR-модели и одновременно относится к направлению применения нейронных сетей в задачах оценки опционов [6].

Проблема исследования заключается в том, что классические численные методы оценки американского пут-опциона не всегда удобны для совместного вычисления цены и греческих символов в едином дифференцируемом вычислительном графе. Возникает необходимость реализовать и проверить архитектуру, которая сохраняет эквивалентность биномиальному дереву, но при этом позволяет применять автоматическое дифференцирование к параметрам модели.

Объектом исследования являются методы оценки опционов. Предметом исследования является нейросетевая архитектура BTNet для оценки американского пут-опциона и вычисления его чувствительностей к рыночным параметрам.

Цель работы состоит в реализации и верификации BTNet для оценки американских пут-опционов и вычисления греческих символов средствами автоматического дифференцирования.

Для достижения указанной цели в работе поставлены следующие задачи:

1. Изучить модели Блэка-Шоулса, Кокса-Росса-Рубинштейна и нейросетевую эквивалентность BTNet.
2. Реализовать программный комплекс `bttn_bs` на языке Python с использованием PyTorch.
3. Верифицировать точность прайсинга европейских и американских пут-опционов относительно эталонных расчетов QuantLib.
4. Вычислить греческие символы Delta, Gamma, Vega и Theta через механизм автоматического дифференцирования.
5. Исследовать влияние переноса весов из европейской модели BTNetEuropean в американскую модель BTNetAmerican на точность цен и греческих символов.

Научная новизна работы заключается в применении дифференцируемой реализации BTNet к задаче оценки американского пут-опциона, а также в анализе

чувствительности греческих символов к аналитической и перенесенной инициализации весов. Отдельное внимание уделяется эффекту нулевой Γ , возникающему из-за кусочно-линейной природы ReLU- и maxout-архитектур.

Практическая значимость работы состоит в создании программного комплекса `bttn_bs`, который позволяет оценивать опционы, сравнивать результаты с QuantLib и вычислять чувствительности цены к рыночным параметрам. Такой инструмент может использоваться в учебных и исследовательских задачах количественных финансов, а также как основа для дальнейшего развития дифференцируемых моделей анализа рыночного риска.

Работа состоит из введения, трех глав, заключения, списка используемых источников и приложений. В первой главе рассматриваются теоретические основы оценки опционов, модели Блэка-Шоулса и CRR, а также нейросетевая эквивалентность BTNet. Во второй главе описывается программная реализация пакета `bttn_bs`, архитектура моделей BTNetEuropean и BTNetAmerican, а также методика верификации с использованием QuantLib. В третьей главе приводятся численные эксперименты, результаты вычисления греческих символов и анализ эксперимента переноса весов из европейской модели в американскую.

Глава 1. Теоретические основы

1.1. Опционы и задача оценки производных финансовых инструментов

Производные финансовые инструменты, или деривативы, представляют собой контракты, стоимость которых определяется динамикой некоторого базового актива. В качестве базового актива могут выступать акции, фондовые индексы, валюты, процентные ставки, сырьевые товары и другие рыночные показатели. Экономическая роль деривативов состоит в перераспределении финансовых рисков между участниками рынка: с их помощью инвесторы могут хеджировать неблагоприятные изменения цен, строить арбитражные стратегии и формировать позиции с заданным профилем доходности.

Опцион является одним из основных видов производных финансовых инструментов. Покупатель опциона получает право, но не обязанность, совершить сделку с базовым активом по заранее установленной цене исполнения K . Если контракт дает право купить базовый актив, он называется опционом колл; если контракт дает право продать базовый актив, он называется опционом пут. Величина выплаты опциона зависит от соотношения между рыночной ценой базового актива и ценой исполнения, поэтому задача оценки опциона сводится к определению справедливой текущей стоимости будущей случайной выплаты.

По условиям исполнения опционы обычно делятся на европейские и американские. Европейский опцион может быть исполнен только в дату истечения T . Американский опцион, напротив, может быть исполнен в любой момент времени до истечения срока действия контракта. Это дополнительное право делает американский опцион не дешевле соответствующего европейского опциона с теми же параметрами, однако существенно усложняет его оценку. Для европейских опционов в рамках классических предположений доступна аналитическая формула Блэка-Шоулса, тогда как для американских опционов необходимо учитывать оптимальный момент досрочного исполнения.

В настоящей работе основное внимание уделяется американскому пут-опциону. Такой контракт дает держателю право продать базовый актив по цене K . Его внутренняя стоимость в момент времени t при цене базового актива S равна $\max(K - S, 0)$. Если цена актива существенно ниже цены исполнения, владелец пут-опциона может получить положительную выплату уже до даты экспирации. Поэтому для

американского пута право досрочного исполнения имеет реальную экономическую ценность и формирует так называемую премию за раннее исполнение.

Экономический смысл американского пут-опциона связан прежде всего с защитой от падения цены базового актива. Инвестор, владеющий активом, может использовать пут-опцион как страхование: при снижении цены актива опцион компенсирует часть убытков. Возможность досрочного исполнения особенно важна при глубоком положении опциона в деньгах, когда немедленное получение выплаты может оказаться выгоднее дальнейшего удержания контракта. В то же время досрочное исполнение не всегда оптимально, поскольку сохранение опциона оставляет владельцу временную стоимость и возможность получить большую выплату в будущем.

С математической точки зрения оценка американского пут-опциона является задачей оптимальной остановки. Держатель опциона выбирает момент исполнения из всего интервала от текущего времени до даты экспирации, стремясь максимизировать дисконтированное математическое ожидание выплаты $\max(K - S_t, 0)$. В каждый момент времени существует область продолжения, где рационально сохранять опцион, и область исполнения, где рационально реализовать право продажи. Граница между этими областями называется границей раннего исполнения и заранее неизвестна.

Именно наличие свободной границы делает американский пут-опцион более сложным объектом по сравнению с европейским опционом. Для него, как правило, не существует простой замкнутой формулы цены, поэтому используются численные методы: биномиальные и триномиальные деревья, конечно-разностные схемы, методы Монте-Карло и их модификации [1; 7]. Среди этих подходов биномиальная модель Кокса-Росса-Рубинштейна занимает особое место, поскольку она имеет прозрачную финансовую интерпретацию и естественным образом реализует выбор между продолжением позиции и досрочным исполнением на каждом узле дерева [4].

1.2. Модели Блэка-Шоулса и Кокса-Росса-Рубинштейна

Классической отправной точкой современной теории оценки опционов является модель Блэка-Шоулса-Мертон [2; 3]. В этой модели предполагается, что цена базового актива S_t следует геометрическому броуновскому движению, то есть описывается стохастическим дифференциальным уравнением $dS_t = \mu S_t dt + \sigma$

$S_t dW_t$, где μ обозначает ожидаемую доходность, σ – постоянную волатильность, а W_t – стандартный винеровский процесс. В риск-нейтральной мере ожидаемая доходность заменяется безрисковой ставкой r , что позволяет оценивать производные инструменты как дисконтированное математическое ожидание будущей выплаты.

Модель Блэка-Шоулса строится на ряде упрощающих предпосылок: непрерывность торгов, отсутствие арбитража, отсутствие транзакционных издержек и налогов, возможность занимать и размещать средства по одной безрисковой ставке, постоянство процентной ставки и волатильности, а также логнормальность распределения цены базового актива. Эти предположения редко выполняются полностью на реальном рынке, однако модель остается фундаментальной, поскольку задает аналитический ориентир для европейских опционов и используется как основа для более сложных моделей.

Для европейского пут-опциона на актив без дивидендов цена в модели Блэка-Шоулса-Мертона задается формулой $P = K \exp(-rT) N(-d_2) - S_0 N(-d_1)$, где $d_1 = (\ln(S_0 / K) + (r + \sigma^2 / 2)T) / (\sigma \sqrt{T})$, $d_2 = d_1 - \sigma \sqrt{T}$, K – цена исполнения, T – время до экспирации, S_0 – текущая цена базового актива, а $N(\cdot)$ – функция распределения стандартной нормальной случайной величины [2; 3]. Формула получается из безарбитражного аргумента и граничного условия $P(S, T) = \max(K - S, 0)$.

Несмотря на важность модели Блэка-Шоулса, она напрямую применима прежде всего к европейским опционам. Для американского пут-опциона формула Блэка-Шоулса не учитывает возможность досрочного исполнения. Если владелец опциона имеет право исполнить контракт до даты T , то стоимость опциона зависит не только от конечной выплаты, но и от оптимальной стратегии выбора момента исполнения. Именно поэтому американский пут обычно оценивается численными методами, а формула Блэка-Шоулса может использоваться только как ориентир для соответствующего европейского опциона.

Одним из наиболее известных численных подходов является биномиальная модель Кокса-Росса-Рубинштейна [4]. В этой модели время до экспирации делится на n равных интервалов длины $\Delta t = T / n$. На каждом шаге цена базового актива может либо увеличиться в u раз, либо уменьшиться в d раз. Для модели CRR

параметры выбираются как $u = \exp(\sigma \sqrt{\Delta t})$ и $d = \exp(-\sigma \sqrt{\Delta t}) = 1 / u$. Такая конструкция обеспечивает рекомбинирующее дерево: после роста и падения цена возвращается в тот же узел, что и после падения и роста.

Переход к риск-нейтральной мере в биномиальной модели задает вероятность роста $p = (\exp(r \Delta t) - d) / (u - d)$ и вероятность падения $q = 1 - p$. Эти величины не являются субъективными прогнозами движения рынка; они подбираются так, чтобы дисконтированная цена базового актива была мартингалом. Благодаря этому стоимость опциона может быть вычислена как дисконтированная ожидаемая стоимость будущей выплаты в риск-нейтральной мере.

Для европейского пут-опциона алгоритм CRR начинается с построения терминальных значений цены актива $S(n, j) = S_0 u^j d^{(n-j)}$, где j – число движений вверх. В конечных узлах дерева задается выплата $V(n, j) = \max(K - S(n, j), 0)$. Затем цена опциона вычисляется обратной индукцией: $V(i, j) = \exp(-r \Delta t) [p V(i+1, j+1) + q V(i+1, j)]$ для $i = n - 1, \dots, 0$. Итоговая цена опциона равна значению $V(0, 0)$ в корневом узле дерева.

Для американского пут-опциона обратная индукция модифицируется за счет сравнения двух альтернатив в каждом узле дерева. Первая альтернатива – удерживать опцион, получая продолженную стоимость $\exp(-r \Delta t) [p V(i+1, j+1) + q V(i+1, j)]$. Вторая альтернатива – немедленно исполнить опцион и получить внутреннюю стоимость $K - S(i, j)$, если она положительна. Поэтому рекуррентное соотношение имеет вид $V(i, j) = \max(K - S(i, j), \exp(-r \Delta t) [p V(i+1, j+1) + q V(i+1, j)])$. Именно операция максимума реализует право раннего исполнения.

Биномиальная модель CRR важна для настоящей работы по двум причинам. Во-первых, она дает понятный и воспроизводимый численный эталон для оценки американского пут-опциона [4]. Во-вторых, ее алгоритм обратной индукции имеет локальную структуру: каждое значение на предыдущем временном слое зависит только от двух соседних значений следующего слоя. Эта структура естественно интерпретируется как одномерная свертка и служит основой для построения нейросетевой архитектуры BTNet [5].

1.3. Нейросетевая эквивалентность BTNet

Нейросетевая эквивалентность BTNet основана на наблюдении, что алгоритм обратной индукции в биномиальной модели имеет локальную и повторяющуюся

структуру. На каждом временном шаге цена опциона в одном узле вычисляется по двум соседним значениям следующего слоя дерева. Такая операция математически совпадает с одномерной сверткой с фильтром размера 2. Следовательно, последовательность шагов обратной индукции может быть представлена как последовательность слоев нейронной сети прямого распространения [5].

В обычном применении нейронные сети рассматриваются как универсальные аппроксиматоры, параметры которых подбираются по данным. В архитектуре BTNet логика другая: структура сети заранее строится из финансовой модели, а веса имеют явный экономический смысл. Такая сеть не является черным ящиком в полном смысле слова, поскольку ее слои соответствуют шагам биномиального дерева, а элементы фильтра соответствуют дисконтированным риск-нейтральным вероятностям [4; 5].

Первый слой BTNet отвечает за формирование терминальных выплат опциона. На вход сети подается цена исполнения K . Далее полносвязный слой DenseLayer формирует вектор из $n + 1$ элементов, соответствующий выплатам в конечных узлах биномиального дерева. Для пут-опциона каждая компонента имеет вид $\max(K - S_j^n, 0)$, где S_j^n - цена базового актива в j -м конечном узле дерева. Именно поэтому в начальном слое используется ReLU: она реализует положительную часть выражения $K - S$.

После формирования терминальных выплат европейская модель BTNetEuropean применяет n сверточных слоев ConvLayer. Каждый ConvLayer берет два соседних значения текущего вектора и вычисляет их взвешенную сумму. Если фильтр слоя равен $W = (\exp(-r \Delta t) q, \exp(-r \Delta t) p)$, где p и q - риск-нейтральные вероятности роста и падения, то один такой слой воспроизводит один шаг обратной индукции CRR для европейского опциона [4; 5].

Таким образом, для европейского пут-опциона прямой проход BTNetEuropean полностью соответствует прохождению биномиального дерева от листьев к корню. После первого слоя размерность вектора равна $n + 1$, после каждого сверточного слоя она уменьшается на единицу, и после n сверточных слоев остается одно число - цена опциона в начальном узле дерева. Это совпадает с результатом классической обратной индукции.

Для американского пут-опциона одной свертки недостаточно, поскольку в каждом узле необходимо сравнивать продолженную стоимость и стоимость немедленного исполнения. Поэтому архитектура BTNetAmerican вместо обычных сверточных слоев использует MaxoutLayer. Такой слой содержит две ветви: первая ветвь вычисляет дисконтированную ожидаемую стоимость продолжения, а вторая вычисляет внутреннюю стоимость $K - S_{i,j}$ в соответствующем узле дерева. Выходом слоя является поэлементный максимум между этими величинами.

Операция maxout в BTNetAmerican имеет прямую финансовую интерпретацию. Если продолженная стоимость больше внутренней стоимости, оптимально не исполнять опцион и перейти к следующему периоду. Если внутренняя стоимость больше, рациональный владелец опциона реализует право продажи немедленно. Следовательно, maxout реализует правило оптимального досрочного исполнения, а прямой проход сети становится эквивалентным обратной индукции для американского пут-опциона [5].

Аналитическая инициализация весов является ключевым элементом BTNet. В отличие от произвольной нейронной сети, здесь веса не обязательно искать градиентным спуском. Для заданных параметров рынка S_0 , σ , r , T и глубины дерева n можно сразу вычислить $u = \exp(\sigma \sqrt{\Delta t})$, $d = \exp(-\sigma \sqrt{\Delta t})$, $p = (\exp(r \Delta t) - d) / (u - d)$ и $q = 1 - p$. Эти величины задают как геометрию дерева, так и фильтр сверточных слоев [4].

Финансовый смысл такой инициализации состоит в том, что сеть наследует безарбитражную риск-нейтральную структуру CRR. Фильтр W не является произвольным набором чисел: он задает дисконтированное математическое ожидание будущей стоимости опциона. Поэтому при аналитической инициализации BTNet не просто обучается на примерах, а конструктивно воспроизводит известный численный метод оценки опционов [5].

В таблице 1.1 приведено соответствие между элементами биномиальной модели и элементами архитектуры BTNet. Терминальные выплаты соответствуют выходу DenseLayer, шаг обратной индукции - ConvLayer, условие раннего исполнения - MaxoutLayer, а итоговая цена опциона - выходному нейрону после последовательного уменьшения размерности вектора.

Таблица 1.1 – Соответствие элементов CRR-модели и архитектуры BTNet

Элемент CRR-модели	Элемент BTNet	Финансовый смысл
Терминальная выплата $\max(K - S, 0)$	DenseLayer + ReLU	Формирование выплат в листьях дерева
Дисконтированное ожидание	ConvLayer	Один шаг обратной индукции
Сравнение продолжения и исполнения	MaxoutLayer	Учет права раннего исполнения
Корневой узел дерева	Последний выход сети	Текущая цена опциона

Роль ReLU, одномерной свертки и maxout в этой архитектуре различна. ReLU кодирует положительную часть выплаты пут-опциона. Одномерная свертка реализует локальный переход от двух будущих узлов к одному предыдущему узлу. Maxout добавляет оптимизационное решение, необходимое для американского опциона. Вместе эти операции позволяют построить сеть, которая остается простой с точки зрения машинного обучения, но сохраняет финансовую структуру исходной модели.

Важное преимущество BTNet состоит в совместимости с автоматическим дифференцированием. Если вычисления записаны в виде операций PyTorch, то цена опциона становится частью вычислительного графа, и производные по параметрам могут быть получены через `torch.autograd.grad` [8]. Это особенно важно для расчета греческих символов, поскольку в классическом биномиальном дереве их часто приходится находить конечными разностями.

При этом архитектура BTNet имеет и ограничения. Поскольку ReLU и maxout являются кусочно-линейными операциями, цена опциона как функция входных параметров также может быть кусочно-линейной. Это полезно для интерпретируемости и устойчивости вычислений, но приводит к эффекту нулевой второй производной почти всюду. В дальнейшем это проявляется в поведении Gamma и требует отдельного анализа в экспериментальной части работы.

Таким образом, нейросетевая эквивалентность BTNet не сводится к простой аппроксимации цен. Она показывает, что известный численный метод может быть представлен как специальная нейронная архитектура с заранее интерпретируемыми весами. Для настоящей работы это принципиально: BTNetAmerican позволяет одновременно сохранять логику CRR для американского пут-опциона и использовать инструменты PyTorch для вычисления чувствительностей.

Сравнение с существующими решениями

Для обоснования выбора VTNet важно сопоставить рассматриваемую архитектуру с другими подходами к оценке опционов. В литературе и практических библиотеках используются несколько классов решений: аналитические формулы, решеточные методы, конечно-разностные схемы, методы Монте-Карло и нейросетевые аппроксимации. Каждый класс методов решает свою часть задачи: одни дают высокую интерпретируемость, другие лучше масштабируются на сложные контракты, третьи удобны для быстрого приближенного расчета на больших наборах параметров [1].

Аналитическая формула Блэка-Шоулса-Мертона является фундаментальным ориентиром для оценки европейских опционов [2; 3]. Ее преимущество состоит в замкнутой форме решения: цена вычисляется быстро, воспроизводимо и без численной схемы по времени. Кроме того, для европейского опциона доступны аналитические формулы греческих символов, что делает модель удобной для обучения и проверки программных реализаций. Именно поэтому в настоящей работе формула Блэка-Шоулса используется как эталон для VTNetEuropean и как источник обучающих цен в эксперименте переноса весов.

Однако аналитический подход имеет принципиальное ограничение: он не дает универсального решения для американского пут-опциона. Право досрочного исполнения делает цену зависящей от оптимальной стратегии остановки, а граница исполнения заранее неизвестна. Поэтому попытка применять только замкнутые формулы приводит либо к упрощениям, либо к приближенным решениям, которые не всегда удобны для проверки архитектуры, построенной на явной обратной индукции. Для данной работы важно не просто получить цену, а сохранить структуру решения по дереву, чтобы затем анализировать чувствительности и влияние весов.

Биномиальная модель Кокса-Росса-Рубинштейна является более близкой к VTNet по внутренней логике [4]. В CRR время до экспирации разбивается на конечное число шагов, а в каждом узле цена опциона вычисляется через дисконтированное риск-нейтральное ожидание будущих значений. Для американского опциона в узле дополнительно берется максимум между продолженной стоимостью и внутренней стоимостью немедленного исполнения. Эта рекуррентная структура естественно соответствует DenseLayer, ConvLayer и MaxoutLayer в VTNet.

Главное достоинство CRR состоит в прозрачности. Каждый параметр имеет финансовый смысл: u и d описывают возможное движение базового актива, p задает риск-нейтральную вероятность, а дисконтирование отражает стоимость денег во времени. Поэтому CRR хорошо подходит для учебных и исследовательских задач, где важно видеть механизм оценки. Недостаток состоит в том, что при большом числе шагов и многократном пересчете по сетке параметров классическая реализация может быть менее удобна для включения в современные дифференцируемые вычислительные графы.

Конечно-разностные методы решают связанную с опционом дифференциальную задачу на сетке по времени и цене базового актива [1]. Они особенно полезны для контрактов, где нужно аккуратно аппроксимировать свободную границу или учитывать более сложные динамики. В то же время такие методы требуют выбора схемы, сетки, граничных условий и критериев устойчивости. Для целей настоящей работы конечно-разностный подход был бы менее прямым: он хорошо подходит для численного решения уравнения, но хуже демонстрирует нейросетевую эквивалентность именно биномиальной обратной индукции.

Методы Монте-Карло широко применяются для оценки производных инструментов, особенно когда размерность задачи возрастает. Для американских опционов классическим решением является метод Longstaff–Schwartz, в котором задача досрочного исполнения приближается с помощью регрессии продолженной стоимости по смоделированным траекториям [7]. Его сила состоит в применимости к высокоразмерным задачам и сложным источникам неопределенности. Однако результат зависит от числа траекторий, выбора базисных функций и статистической ошибки, что делает метод менее удобным как точный эталон для небольшого одномерного эксперимента.

Нейросетевые методы оценки опционов развиваются как способ ускорить прайсинг после обучения модели. Например, в работе Liu, Oosterlee и Bohte нейронные сети используются для аппроксимации цен опционов и вычисления подразумеваемой волатильности [6]. Такой подход полезен, когда требуется быстро вычислять значения на большом наборе рыночных параметров. Но обычная нейронная сеть часто выступает как универсальный аппроксиматор: она может

давать хорошую точность, но ее веса не всегда имеют прямую финансовую интерпретацию.

BTNet занимает промежуточное положение между классическими численными методами и нейросетевыми аппроксимациями. С одной стороны, архитектура реализуется как нейронная сеть PyTorch и совместима с автоматическим дифференцированием [8]. С другой стороны, она не является произвольной многослойной сетью: ее слои соответствуют конкретным операциям CRR-дерева [5]. Поэтому BTNet сохраняет интерпретируемость, но получает инженерные преимущества нейросетевого представления: батчевые вычисления, использование autograd и возможность экспериментов с обучаемыми параметрами.

Таблица 1.2 – Сравнение подходов к оценке опционов

Подход	Основная идея	Преимущества	Ограничения для данной работы
Формула Блэка-Шоулса-Мертона	Замкнутая цена европейского опциона в непрерывной модели	Быстрый расчет, аналитические греки, удобный эталон	Не дает универсальной цены американского пута
Биномиальная модель CRR	Обратная индукция по дереву риск-нейтральных цен	Прозрачная финансовая интерпретация, естественное раннее исполнение	При большом числе шагов требует многократного численного пересчета
Конечно-разностные методы	Решение уравнения ценообразования на сетке	Гибкость для свободной границы и сложных условий	Нужно выбирать сетку, схему и условия устойчивости
Longstaff–Schwartz	Монте-Карло с регрессией продолженной стоимости	Подходит для высокоразмерных и сложных задач	Есть статистическая ошибка и зависимость от базисных функций

Обычные нейронные сети	Аппроксимация цены по параметрам опциона	Быстрый расчет после обучения, хорошая масштабируемость	Ограниченная интерпретируемость весов
BTNet	Нейросетевая форма CRR-обратной индукции	Интерпретируемые веса, autograd, естественная модель раннего исполнения	Кусочно-линейность ограничивает Gamma; точность зависит от глубины дерева

Сравнение показывает, что BTNet не заменяет классические методы полностью, а занимает особую нишу. В отличие от формулы Блэка-Шоулса, он может описывать американский пут через раннее исполнение. В отличие от обычного CRR-кода, он сразу записан в форме вычислительного графа. В отличие от универсальных нейронных сетей, он сохраняет финансовый смысл весов. Именно сочетание этих свойств делает BTNet подходящим объектом для данной ВКР: задача состоит не только в аппроксимации цены, но и в проверке интерпретируемой дифференцируемой архитектуры.

С точки зрения вычисления греков различия между подходами особенно заметны. В модели Блэка-Шоулса греки выводятся аналитически, но это преимущество распространяется прежде всего на европейские опционы и стандартные предположения о динамике базового актива. В классической CRR-модели греки обычно получают конечными разностями: цена пересчитывается при малом изменении S_0 , σ или T . Такой способ прост, но требует нескольких дополнительных прогонов модели и чувствителен к выбору шага конечной разности.

В BTNet появляется другой путь: если параметры включены в вычислительный граф, чувствительности можно получать через автоматическое дифференцирование. Это особенно важно для исследовательской постановки, где требуется анализировать не только цену, но и влияние архитектуры на Delta, Vega, Theta и Gamma. При этом работа показывает, что автоматическое дифференцирование не является универсальным решением всех проблем: кусочно-линейные операции ReLU и maxout делают Gamma равной нулю почти всюду, поэтому архитектура требует осторожной финансовой интерпретации.

Существующие нейросетевые подходы к оценке опционов часто ориентированы на приближение отображения от параметров контракта к цене. Такой подход может быть эффективен в задачах ускорения вычислений, но возникает вопрос о надежности вне обучающей области и о поведении производных. Если сеть обучена только на ценах, ее греки могут быть менее устойчивыми, чем сама цена. В настоящей работе этот вопрос проявляется в эксперименте переноса весов: европейское обучение дает фильтр, который немного улучшает одну постановку, но ухудшает американскую цену и чувствительности.

Важное отличие BTNet от обычной аппроксимационной сети состоит в наличии аналитической инициализации. Веса DenseLayer и ConvLayer можно задать напрямую из параметров CRR, поэтому модель начинает работу не с произвольных случайных коэффициентов, а с финансово осмысленного решения. Это снижает риск того, что точность будет достигнута ценой потери интерпретируемости. Для американского пут-опциона данное свойство особенно важно, поскольку решение о раннем исполнении чувствительно к локальным изменениям продолженной стоимости.

Библиотека QuantLib в этой системе играет роль практического эталона, а не конкурирующего нейросетевого метода [10]. Она содержит проверенные реализации аналитических и численных моделей, в том числе биномиальные движки для ванильных опционов. Поэтому сравнение с QuantLib позволяет отделить две ошибки: ошибку программной реализации BTNet и ошибку конечной глубины дерева. Если BTNet с аналитическими весами близок к QuantLib CRR с большим числом шагов, это подтверждает корректность как математической идеи, так и программной реализации.

Сравнение существующих решений также объясняет, почему в работе не ставится задача заменить все методы оценки опционов одной нейросетью. Формула Блэка-Шоулса остается лучшим инструментом для стандартного европейского опциона, CRR остается естественным численным методом для одномерного американского опциона, Longstaff–Schwartz полезен для многомерных задач, а универсальные нейросети удобны для быстрой аппроксимации сложных поверхностей цен. Цель BTNet уже: показать, что известный численный алгоритм можно представить как интерпретируемую дифференцируемую архитектуру.

Таблица 1.3 – Позиционирование BTNet относительно близких решений

Критерий	CRR	Обычная нейронная сеть	BTNet
Финансовая интерпретация весов	Высокая: веса задаются риск-нейтральной мерой	Обычно низкая: веса являются параметрами аппроксимации	Высокая при аналитической инициализации
Американское раннее исполнение	Естественно реализуется через максимум	Требует специальной постановки обучения	Реализуется через MaxoutLayer
Автоматическое дифференцирование	Не является базовой формой метода	Доступно в нейросетевых фреймворках	Доступно при функциональной записи CRR/BTNet
Проверяемость результата	Сравнение с более глубоким деревом	Сравнение с тестовыми данными	Сравнение с CRR и QuantLib
Основной риск	Ошибка дискретизации	Потеря интерпретируемости и ошибки вне обучающей области	Кусочно-линейность и зависимость от глубины дерева

Таким образом, обзор существующих решений показывает, что выбор BTNet для настоящего исследования является осознанным компромиссом. Архитектура наследует прозрачность CRR, но реализуется в инструментarii нейронных сетей. Она не устраняет все ограничения классической модели: конечная глубина дерева по-прежнему влияет на точность, а кусочно-линейные операции ограничивают поведение Gamma. Тем не менее именно эта комбинация делает BTNet удобной основой для изучения американского пут-опциона, автоматического дифференцирования и эксперимента с переносом весов.

Сравнение научных работ и программных решений

Сравнение близких научных работ позволяет точнее сформулировать место настоящего исследования. Классические работы Блэка, Шоулса и Мертона заложили непрерывную теорию безарбитражной оценки европейских опционов [2; 3]. Их вклад состоит не только в конкретной формуле цены, но и в самой идее риск-

нейтральной оценки: справедливая стоимость производного инструмента определяется не индивидуальным ожиданием инвестора, а возможностью построить самофинансируемый хеджирующий портфель. Для настоящей работы эта линия важна как теоретическая основа и как источник аналитического эталона для европейского пута.

Работа Кокса, Росса и Рубинштейна предлагает дискретную альтернативу непрерывной модели [4]. CRR-дерево проще для программной реализации и особенно удобно для объяснения раннего исполнения: на каждом узле дерево хранит не только возможную цену актива, но и локальное решение о продолжении или исполнении. Именно поэтому CRR является ближайшим классическим предшественником BTNet. Разница между ними не в финансовой логике, а в форме записи: CRR обычно описывается как рекурсивный численный алгоритм, тогда как BTNet представляет ту же рекурсию как композицию нейросетевых слоев.

Учебная и справочная литература, например работа Hull, систематизирует широкий класс методов оценки опционов: аналитические решения, деревья, конечно-разностные методы, Монте-Карло и приближения для американских контрактов [1]. Для ВКР такая литература важна как рамка сравнения: она показывает, что не существует одного метода, оптимального во всех случаях. Европейские опционы удобно считать аналитически, одномерные американские опционы хорошо описываются деревьями, а высокоразмерные контракты часто требуют симуляционных методов. Следовательно, выбор BTNet должен оцениваться не абстрактно, а относительно конкретной задачи: американский пут, дифференцируемость и интерпретируемые веса.

Метод Longstaff–Schwartz решает проблему американского исполнения в симуляционной постановке [7]. Его идея состоит в том, чтобы на смоделированных траекториях приближать продолженную стоимость регрессией и сравнивать ее с немедленной выплатой. Этот подход стал важным инструментом для задач, где дерево становится слишком большим или плохо масштабируется по размерности. Однако в одномерной постановке настоящей работы Longstaff–Schwartz не является главным эталоном: он содержит статистическую ошибку, зависит от выбора регрессионных признаков и не дает такого прозрачного соответствия слоям BTNet, как CRR.

Нейросетевые работы, такие как исследование Liu, Oosterlee и Bohte, демонстрируют другую мотивацию: после обучения сеть может быстро приближать цены опционов и подразумеваемые волатильности [6]. Это направление важно для практики, потому что в реальных системах часто требуется многократно пересчитывать цены на больших наборах параметров. Но такая постановка обычно рассматривает сеть как аппроксиматор функции цены. Настоящая работа отличается тем, что BTNet не просто обучается на данных, а имеет заранее заданную структуру, соответствующую финансовому алгоритму.

Работа Шорохова является наиболее близкой к настоящей ВКР, поскольку именно в ней рассматривается представление биномиального дерева как нейронной сети [5]. В этой постановке терминальная выплата задается первым слоем, а обратная индукция реализуется последовательностью сверточных операций. Настоящая работа развивает эту идею в сторону американского пут-опциона, вычисления греческих символов и эксперимента с переносом весов. Таким образом, вклад ВКР состоит не в замене исходной теоремы, а в ее программной реализации, проверке на эталоне QuantLib и анализе чувствительностей.

Отдельного упоминания заслуживает QuantLib как программное решение [10]. В отличие от исследовательских нейросетевых моделей, QuantLib является зрелой библиотекой количественных финансов и содержит проверенные реализации большого числа методов. Поэтому в работе она используется не как объект улучшения, а как внешний ориентир качества. Сравнение с QuantLib дисциплинирует эксперимент: если BTNet дает близкие значения при аналитической инициализации, можно говорить, что нейросетевая форма корректно воспроизводит классическую финансовую схему.

Таблица 1.4 – Связь настоящей работы с близкими источниками

Источник или решение	Основной вклад	Связь с настоящей работой
Black, Scholes; Merton	Аналитическая теория европейских опционов и риск-нейтральная оценка	Используется как теоретическая база и эталон для европейского пута
Cox, Ross, Rubinstein	Биномиальное дерево и обратная индукция	Является алгоритмической основой BTNet и

		американского maxout-прохода
Hull	Систематизация методов оценки производных инструментов	Используется для общей классификации методов, греков и ограничений моделей
Longstaff–Schwartz	Симуляционная оценка американских опционов через регрессию	Показывает альтернативный путь для раннего исполнения, особенно в высоких размерностях
Liu, Oosterlee, Bohte	Нейросетевое приближение цен и подразумеваемой волатильности	Сравнивается с BTNet как пример аппроксимационного нейросетевого подхода
Шорохов	Эквивалентность биномиального дерева и нейросетевой архитектуры	Непосредственная теоретическая основа реализации BTNet
QuantLib	Проверенные программные реализации моделей оценки опционов	Используется как внешний эталон для численной верификации

Из таблицы 1.4 видно, что настоящая работа находится на пересечении двух направлений: классической численной оценки опционов и нейросетевых методов. От классических моделей она наследует финансовую интерпретацию, риск-нейтральное дисконтирование и логику раннего исполнения. От нейросетевых методов она наследует форму вычислительного графа и возможность автоматического дифференцирования. Такое сочетание и определяет исследовательскую ценность BTNet для задач анализа риска.

Важное отличие настоящей работы от чисто прикладного использования QuantLib состоит в том, что здесь рассматривается не только получение цены, но и внутренняя структура вычисления. QuantLib способен дать надежный численный результат, однако его использование само по себе не отвечает на вопрос, можно ли

представить алгоритм в виде нейросети с интерпретируемыми весами. BTNet делает этот вопрос центральным: цена становится результатом прямого прохода сети, а каждый слой соответствует понятному финансовому шагу.

Отличие от обычных нейросетевых аппроксимаций также существенно. Если сеть обучается на наборе цен, она может хорошо интерполировать в обучающей области, но ее поведение при изменении параметров зависит от качества данных, архитектуры и регуляризации. BTNet с аналитической инициализацией изначально строится как корректный финансовый алгоритм. Поэтому в работе проверяется не только точность цены, но и устойчивость финансового смысла весов при попытке переноса из европейской задачи в американскую.

Именно сравнительный анализ показывает, почему эксперимент с переносом весов является содержательным. Если рассматривать BTNet как обычную нейронную сеть, то перенос весов кажется естественным приемом: модель, обученная на европейском опционе, должна содержать полезные признаки для близкой американской задачи. Если же рассматривать BTNet как форму CRR, то становится понятно, что фильтр W имеет конкретный смысл дисконтированного риск-нейтрального ожидания. Поэтому любое обучение, которое смещает W от CRR-значений, может ухудшить решение о раннем исполнении.

Таким образом, обзор существующих работ не только расширяет теоретическую часть, но и напрямую связан с численными экспериментами. Он объясняет выбор Блэка-Шоулса как эталона для европейской модели, CRR и QuantLib как эталонов для американского прайсинга, Longstaff–Schwartz как альтернативного подхода к раннему исполнению и нейросетевые аппроксимации как близкое, но не тождественное направление. На этом фоне BTNet занимает ясную позицию: это не универсальный черный ящик, а интерпретируемая дифференцируемая запись классического дерева.

Глава 2. Реализация программного комплекса `bttn_bs`

2.1. Архитектура пакета и вычислительная среда

Практическая часть работы выполнена в виде программного комплекса `bttn_bs`, реализованного на языке Python. Пакет предназначен для построения, обучения и верификации нейросетевых моделей BTNet, которые воспроизводят вычисления биномиальной модели Кокса-Росса-Рубинштейна для европейских и американских пут-опционов. Структура комплекса ориентирована на разделение математической логики, архитектур нейронных сетей, процедур обучения, вычисления греческих символов и внешней верификации через библиотеку QuantLib.

Корневой каталог пакета `bttn_bs` содержит несколько функциональных модулей. Модуль `layers.py` определяет базовые слои, из которых строятся архитектуры BTNet: `DenseLayer`, `ConvLayer` и `MaxoutLayer`. `DenseLayer` используется для формирования терминальных выплат в узлах последнего слоя дерева, `ConvLayer` реализует один шаг дисконтированного усреднения в биномиальной модели, а `MaxoutLayer` добавляет ветвь немедленного исполнения и выполняет операцию максимума, необходимую для американского опциона.

Модуль `model_european.py` содержит класс `BTNetEuropean`. Эта модель принимает на вход цену исполнения K и возвращает оценку европейского пут-опциона. Ее структура соответствует теореме об эквивалентности европейского BTNet и CRR-дерева: начальный полносвязный слой формирует выплаты в конечных узлах, после чего последовательность сверточных слоев выполняет обратную индукцию без учета раннего исполнения.

Модуль `model_american.py` содержит класс `BTNetAmerican`, который является основной моделью настоящей работы. В отличие от европейской архитектуры, после начального слоя используются `MaxoutLayer`. Каждый такой слой вычисляет две величины: продолженную стоимость опциона и стоимость немедленного исполнения. Поэлементный максимум между ними соответствует рекуррентному соотношению для американского пут-опциона в модели CRR.

Модуль `analytics.py` содержит вспомогательные аналитические и численные функции: формулу цены европейского пут-опциона в модели Блэка-Шоулса и реализацию биномиального прайсинга американского пута. Эти функции используются

для генерации обучающих и тестовых данных, а также для первичной проверки корректности результатов.

Модуль `training.py` реализует функцию `train_BTNet`. В ней используется стандартный цикл обучения PyTorch: прямой проход модели, вычисление среднеквадратичной ошибки, обратное распространение градиента и шаг оптимизатора Adam [8; 9]. Такая функция позволяет обучать параметры BTNet на синтетических или эталонных ценах опционов, а также проводить эксперименты с аналитической и перенесенной инициализацией весов.

Модуль `greeks.py` отвечает за вычисление греческих символов. В нем реализован функциональный дифференцируемый проход CRR/BTNet, в котором рыночные параметры S_0 , σ , r и T могут рассматриваться как тензоры PyTorch с `requires_grad=True`. Это позволяет получать Delta, Gamma, Vega и Theta через механизм автоматического дифференцирования, а для американского опциона дополнительно сравнивать результаты с центральными конечными разностями.

Отдельный подмодуль `quantlib` содержит интеграцию с QuantLib. В нем реализованы функции для оценки европейского пута аналитической моделью QuantLib и американского пута с помощью `BinomialVanillaEngine` в схеме CRR. В настоящей работе QuantLib используется не как часть BTNet-модели, а как внешний эталон для проверки точности прайсинга [10].

Вычислительная среда строится на библиотеках PyTorch, NumPy, SciPy, Matplotlib и QuantLib. PyTorch используется для построения нейросетевых слоев и автоматического дифференцирования [8], NumPy - для подготовки массивов данных и численных расчетов, SciPy - для аналитических функций распределения [11], Matplotlib - для построения графиков [12], а QuantLib - для эталонной оценки опционов [10].

Базовый вычислительный эксперимент проводится при следующих параметрах: начальная цена базового актива $S_0 = 0.5$, срок до экспирации $T = 1$, безрисковая ставка $r = 0.05$, волатильность $\sigma = 0.25$ и глубина биномиального дерева $n = 9$. Страйки рассматриваются на интервале от 0.25 до 0.75. Такой набор параметров соответствует экспериментам из исходной теоретической работы по BTNet и обеспечивает удобную воспроизводимость результатов.

Выбор глубины $n = 9$ означает, что нейросетевая архитектура имеет конечное число слоев и воспроизводит CRR-дерево той же глубины. При сравнении с QuantLib в качестве более точного численного ориентира используется CRR-дерево с 500 шагами. Поэтому наблюдаемая ошибка BTNet относительно QuantLib отражает главным образом разницу в дискретизации дерева, а не дополнительную ошибку нейросетевой реализации.

2.2. Реализация моделей BTNetEuropean и BTNetAmerican

Реализация моделей BTNetEuropean и BTNetAmerican основана на прямом соответствии между шагами биномиального дерева CRR и слоями нейронной сети. В программном комплексе это соответствие вынесено в отдельные классы слоев, что позволяет использовать одну и ту же базовую логику для европейского и американского опциона. Такой подход делает код компактным, но сохраняет финансовую интерпретацию каждого вычислительного блока [4; 5].

Базовым строительным элементом является DenseLayer. В контексте BTNet этот слой не используется как произвольный полносвязный слой общего назначения. Его задача состоит в том, чтобы по входному значению цены исполнения K построить вектор терминальных выплат в конечных узлах биномиального дерева. Для пут-опциона каждая компонента такого вектора имеет вид $\max(K - S_j, 0)$. Поэтому после линейного преобразования применяется ReLU, реализующая положительную часть выплаты.

Веса DenseLayer при аналитической инициализации имеют простой вид: все коэффициенты при K равны единице, а смещения равны отрицательным значениям цен базового актива в конечных узлах дерева. В результате выражение внутри ReLU принимает вид $K - S_j$. Это означает, что первый слой сети не обучает произвольное представление данных, а буквально кодирует граничное условие опционного контракта в момент экспирации.

Второй ключевой слой - ConvLayer. Он реализован как одномерная свертка с ядром размера 2 без свободного смещения. На каждом шаге такой слой берет два соседних значения вектора и вычисляет их взвешенную сумму. При весах, равных дисконтированным риск-нейтральным вероятностям, эта операция совпадает с одним шагом обратной индукции CRR: из двух будущих узлов дерева получается одно значение в предыдущем узле [4].

Для европейского пут-опциона после DenseLayer применяется последовательность из n одинаковых по смыслу сверточных шагов. Каждый шаг уменьшает размерность вектора на единицу: из $n + 1$ терминальных выплат получается n значений, затем $n - 1$ и так далее, пока не остается одно число. Это число и является оценкой текущей стоимости европейского пут-опциона. Именно так устроен класс BTNetEuropean.

Общая схема прямого прохода BTNetEuropean приведена на рисунке 2.1. На ней видно, что европейская архитектура состоит из терминального слоя выплат и последовательности одинаковых сверточных шагов обратной индукции.

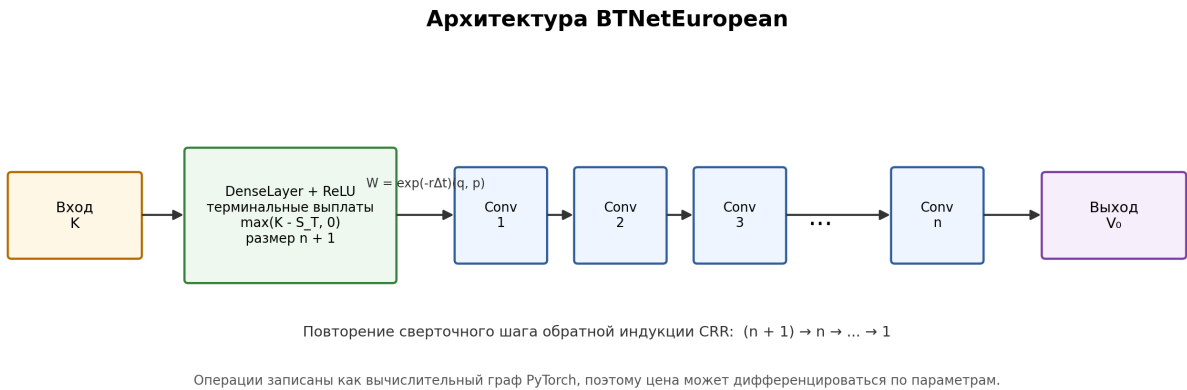


Рисунок 2.1 – Архитектура модели BTNetEuropean

Таблица 2.1 – Основные классы программной реализации BTNet

Класс / модуль	Назначение	Финансовая интерпретация
DenseLayer	Формирует вектор выплат из входного страйка K	Терминальное условие $\max(K - S, 0)$
ConvLayer	Выполняет 1D-свертку с фильтром размера 2	Дисконтированное риск-нейтральное ожидание
MaxoutLayer	Сравнивает продолжение и немедленное исполнение	Условие раннего исполнения американского опциона
BTNetEuropean	Собирает DenseLayer и n ConvLayer	CRR-оценка европейского пут-опциона
BTNetAmerican	Собирает DenseLayer и n MaxoutLayer	CRR-оценка американского пут-опциона

Класс `BTNetEuropean` наследуется от `nn.Module` и содержит два основных компонента: начальный `DenseLayer` и сверточный слой `ConvLayer`, применяемый последовательно n раз. В конструкторе модели задаются параметры S_0 , σ , T , t_0 , r и n_dim . На их основе вычисляется шаг времени Δt , коэффициенты роста и падения u и d , а также фильтр W , соответствующий дисконтированным риск-нейтральным вероятностям.

Метод `forward` в `BTNetEuropean` принимает тензор страйков K . Если вход имеет одномерный вид, он приводится к форме $batch_size \times 1$. Затем значение передается в начальный слой, который формирует вектор терминальных выплат. После этого цикл из n применений `ConvLayer` реализует обратную индукцию по дереву. Благодаря тому, что все операции являются операциями `PyTorch`, модель может работать как с одиночным страйком, так и с батчем значений.

`BTNetAmerican` построен похожим образом, но вместо повторного применения одного сверточного слоя использует список `MaxoutLayer`. Это связано с тем, что на каждом временном слое дерева меняется набор узлов, в которых нужно вычислять стоимость немедленного исполнения. Поэтому для каждого шага создается отдельный `MaxoutLayer` со своим линейным преобразованием ветви исполнения и общей логикой свертки ветви продолжения.

В `MaxoutLayer` первая ветвь вычисляет продолженную стоимость: она применяет одномерную свертку к значениям предыдущего слоя. Вторая ветвь применяет линейное преобразование к цене исполнения K и получает значения $K - S_{i,j}$ для текущих узлов дерева. Затем слой возвращает `torch.max` от двух ветвей. Эта операция является прямым аналогом формулы $V(i,j) = \max(K - S(i,j), continuation)$, используемой в CRR для американского пут-опциона [4; 5].

Архитектура `BTNetAmerican` показана на рисунке 2.2. В отличие от европейской модели, каждый скрытый слой содержит две ветви: ветвь продолжения и ветвь немедленного исполнения, после чего применяется операция максимума.

Архитектура BTNetAmerican

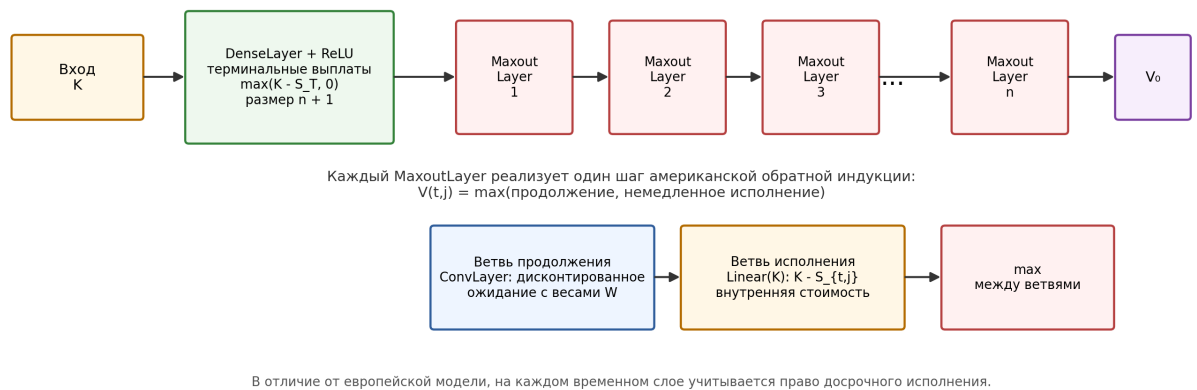


Рисунок 2.2 – Архитектура модели BTNetAmerican

Важная особенность BTNetAmerican состоит в том, что размерность вектора также уменьшается на единицу на каждом шаге. Сначала DenseLayer формирует $n + 1$ терминальных выплат, затем первый MaxoutLayer возвращает n значений, следующий - $n - 1$ и так далее. Последний слой возвращает одно значение, соответствующее текущей цене американского пут-опциона в корне дерева.

Аналитическая инициализация в обеих моделях задается в конструкторах. Для терминального слоя веса равны единице, а смещения кодируют цены актива в узлах дерева. Для сверточной ветви используется фильтр W . Для американской модели дополнительно задаются линейные ветви исполнения на каждом временном слое. Такой способ инициализации позволяет получить финансово корректную модель без предварительного обучения.

Тем не менее пакет поддерживает и обучаемый режим. Поскольку классы наследуются от `nn.Module`, их параметры могут оптимизироваться стандартными средствами PyTorch. В работе для этого используется функция `train_BTNet`, где применяется оптимизатор Adam и среднеквадратичная функция потерь `MSELoss` [8; 9]. Обучение полезно для экспериментов с переносом весов и для проверки, как сильно данные могут сместить параметры относительно аналитических CRR-значений.

Функция `train_BTNet` устроена стандартно для PyTorch. На каждой эпохе модель переводится в режим `train`, обнуляются градиенты оптимизатора, выполняется прямой проход, вычисляется MSE между предсказанными и эталонными ценами, после чего вызывается `backward` и `optimizer.step`. История значений функции потерь сохраняется для последующей визуализации и анализа сходимости.

Для воспроизводимости экспериментов важно, что входные данные имеют фиксированную структуру. Обучающая выборка формируется как набор страйков на интервале от 0.25 до 0.75. Для европейской модели эталонные цены рассчитываются по формуле Блэка-Шоулса-Мертона [2; 3]. Для американской модели при необходимости используются цены, полученные биномиальной моделью или QuantLib. Это разделение позволяет явно понимать, на какой задаче обучалась каждая архитектура.

С точки зрения программной инженерии разделение на `layers.py`, `model_european.py`, `model_american.py` и `training.py` делает реализацию проверяемой. Базовые слои можно анализировать отдельно, модели - сравнивать с теоретическими формулами, а функцию обучения - применять к обеим архитектурам. Такая модульность важна для дальнейшего расширения пакета: например, для добавления гладких активаций, альтернативных деревьев или дополнительных типов опционов.

Итак, реализация `BTNetEuropean` и `BTNetAmerican` в пакете `bttn_bs` сохраняет ключевую идею теоретической модели: нейросеть не заменяет финансовую модель произвольной аппроксимацией, а кодирует ее алгоритм в виде дифференцируемого вычислительного графа. Это создает основу для последующей верификации цен, сравнения с QuantLib и вычисления греческих символов.

2.3. Интеграция QuantLib и методика верификации

Верификация программной реализации необходима по двум причинам. Во-первых, архитектура `BTNet` строится как нейросетевая форма биномиального дерева, поэтому требуется проверить, что прямой проход действительно воспроизводит финансовую логику CRR-модели. Во-вторых, последующие эксперименты с греками и переносом весов имеют смысл только в том случае, если базовая цена опциона согласована с внешним эталоном. В качестве такого эталона в работе используется библиотека QuantLib, широко применяемая в задачах количественных финансов и содержащая готовые реализации аналитических и численных методов оценки производных инструментов [10].

Для европейского пут-опциона эталон задается аналитической формулой Блэка-Шоулса-Мертона [2; 3]. Дополнительно используется аналитическая модель QuantLib, что позволяет отделить возможные ошибки собственной реализации формулы от ошибок нейросетевой архитектуры. Если цена `BTNetEuropean` близка к аналитическому значению, то корректно реализованы терминальные выплаты,

риск-нейтральное дисконтирование и последовательность переходов по дереву. Небольшое отличие от непрерывной формулы допустимо, поскольку BTNet в базовом эксперименте имеет конечную глубину $n = 9$.

Для американского пут-опциона ситуация принципиально иная: замкнутой аналитической формулы для общего случая нет, поскольку стоимость зависит от оптимальной стратегии досрочного исполнения. Поэтому эталоном выбирается биномиальная CRR-модель QuantLib BinomialVanillaEngine с 500 шагами [4; 10]. Такое дерево существенно глубже, чем BTNet, и рассматривается как более точное численное приближение. Сравнение с ним проверяет не только программную корректность модели, но и размер ошибки, возникающей из-за грубой временной дискретизации.

Интеграция с QuantLib вынесена в подмодуль quantlib пакета bttn_bs. В нем реализованы функции построения процесса BlackScholesMertonProcess, создания платежной функции PlainVanillaPayoff и выбора типа исполнения. Для европейского опциона используется EuropeanExercise, а для американского - AmericanExercise на интервале от начальной даты до даты экспирации. Такой способ организации кода делает эталонные расчеты независимыми от классов BTNetEuropean и BTNetAmerican.

Во всех экспериментах фиксируются рыночные параметры $S_0 = 0.5$, $T = 1$, $r = 0.05$, $\sigma = 0.25$ и глубина BTNet $n = 9$. Цена исполнения K изменяется на равномерной сетке от 0.25 до 0.75. Такой диапазон выбран не случайно: при малых K пут-опцион находится вне денег, при K около S_0 - около денег, а при больших K - в деньгах. Следовательно, одна тестовая сетка охватывает разные режимы поведения цены и позволяет увидеть, где ошибка модели максимальна.

Для каждого значения K вычисляются три величины: цена модели BTNet, цена внешнего эталона и ошибка. Далее строится вектор ошибок $e_i = V_i^{\text{BTNet}} - V_i^{\text{ref}}$. На его основе рассчитываются средняя абсолютная ошибка MAE, корень из средней квадратичной ошибки RMSE и максимальная абсолютная ошибка $\max|err|$. Эти метрики имеют разный смысл: MAE характеризует средний уровень отклонения, RMSE сильнее реагирует на отдельные крупные ошибки, а $\max|err|$ показывает наихудшее расхождение на сетке.

Таблица 2.2 – Методика верификации моделей BTNet

Проверяемый объект	Эталон	Цель проверки
BTNetEuropean	Формула Блэка-Шоулса-Мертон и аналитическая модель QuantLib	Проверить корректность европейского прайсинга и CRR-дисконтирования
BTNetAmerican с аналитической инициализацией	QuantLib CRR, 500 шагов	Оценить точность американского прайсинга при весах, заданных из CRR-модели
BTNetAmerican с переносом весов	QuantLib CRR, 500 шагов	Проверить влияние весов, обученных на европейском опционе, на задачу раннего исполнения
Греки европейского опциона	Аналитические формулы Блэка-Шоулса и конечные разности	Проверить согласованность автоматического дифференцирования
Греки американского опциона	Центральные конечные разности	Проверить чувствительности при отсутствии замкнутой аналитической формулы

Формально метрики записываются следующим образом: $MAE = (1 / m) \sum_i |e_i|$, $RMSE = \sqrt{(1 / m) \sum_i e_i^2}$, $\max|err| = \max_i |e_i|$, где m - число точек тестовой сетки. Использование сразу трех показателей важно, потому что малая средняя ошибка сама по себе не гарантирует устойчивого поведения модели на всем диапазоне страйков. Для риск-менеджмента особенно опасны локальные выбросы, поскольку они могут исказить оценку позиции именно вблизи границы исполнения или около денег [1].

В работе отдельно сравниваются две версии американской модели. Первая версия использует аналитическую инициализацию, при которой веса слоев задаются напрямую через параметры CRR. Вторая версия использует перенос весов из европейской модели в американскую: начальный слой и сверточный фильтр W берутся из модели BTNetEuropean, предварительно обученной на европейских ценах.

Такое сравнение проверяет гипотезу о том, может ли обучение на более простой европейской задаче дать полезную инициализацию для американского опциона.

Принципиальное требование к методике состоит в том, что обе версии сравниваются на одной и той же тестовой сетке и с одним и тем же эталоном QuantLib. Это исключает ситуацию, когда улучшение метрик возникает из-за изменения данных, шага дерева или параметров рынка. Поэтому различия между аналитической и перенесенной инициализацией интерпретируются именно как следствие изменения весов модели, а не как результат другой постановки эксперимента.

Для воспроизводимости фиксируются параметры рынка, диапазон страйков, число шагов эталонного CRR-дерева, глубина BTNet, начальное значение генератора случайных чисел, размер обучающей выборки, число эпох и параметры оптимизатора Adam [9]. Расчеты вынесены в функции пакета `bttn_bs` и воспроизводятся в ноутбуке `bttn-bs-torch-v4.ipynb`. В результате любой численный показатель в главе 3 можно получить повторным запуском эксперимента без ручной подстановки промежуточных значений.

Глава 3. Численные эксперименты и вычисление греческих символов

3.1. Верификация точности прайсинга

Цель численных экспериментов состоит в том, чтобы проверить, насколько реализованные архитектуры BTNetEuropean и BTNetAmerican воспроизводят эталонные методы оценки опционов. Для европейского пут-опциона в качестве теоретического ориентира используется аналитическая формула Блэка-Шоулса-Мертона [2; 3]. Для американского пут-опциона, для которого в общем случае отсутствует замкнутое решение, применяется биномиальная CRR-модель QuantLib с 500 шагами [4; 10].

Базовый эксперимент проводится при фиксированных параметрах $S_0 = 0.5$, $T = 1$, $r = 0.05$, $\sigma = 0.25$ и глубине BTNet $n = 9$. Тестовая сетка состоит из 100 равномерно расположенных значений цены исполнения K на интервале от 0.25 до 0.75. Такой диапазон охватывает три характерные зоны: опционы вне денег, около денег и в деньгах. Это важно, поскольку ошибка прайсинга обычно распределена неравномерно: около денег и вблизи границы раннего исполнения модель испытывает наибольшую нагрузку.

Для европейского пут-опциона модель BTNetEuropean сравнивается с формулой Блэка-Шоулса. Поскольку BTNetEuropean фактически реализует CRR-дерево конечной глубины, полное совпадение с непрерывной формулой не ожидается. Тем не менее близость цен подтверждает корректность терминального слоя, сверточных переходов и дисконтирования. В классической теории при увеличении числа шагов биномиальная цена сходится к цене Блэка-Шоулса, поэтому наблюдаемое расхождение интерпретируется как ошибка дискретизации [4].

Для американского пут-опциона основной проверкой является сравнение BTNetAmerican с QuantLib CRR. В аналитически инициализированной модели веса задаются напрямую из CRR-параметров, поэтому прямой проход сети должен совпадать с обратной индукцией по дереву той же глубины. Отличие от QuantLib с 500 шагами возникает не из-за нейросетевой формы вычислений, а из-за того, что в базовом эксперименте используется более компактное дерево $n = 9$.

Численные результаты для американского пут-опциона приведены в таблице 3.1. Ошибки рассчитаны на одной и той же сетке страйков относительно QuantLib CRR с 500 шагами. В таблице сравниваются два варианта: аналитическая

инициализация фильтра W и перенос весов из предварительно обученной европейской модели.

Графическое сравнение цен и абсолютных ошибок приведено на рисунке 3.1. Левая часть рисунка показывает близость обеих версий BTNetAmerican к эталону QuantLib CRR, а правая часть позволяет увидеть, что перенесенный фильтр W дает более крупные локальные ошибки.

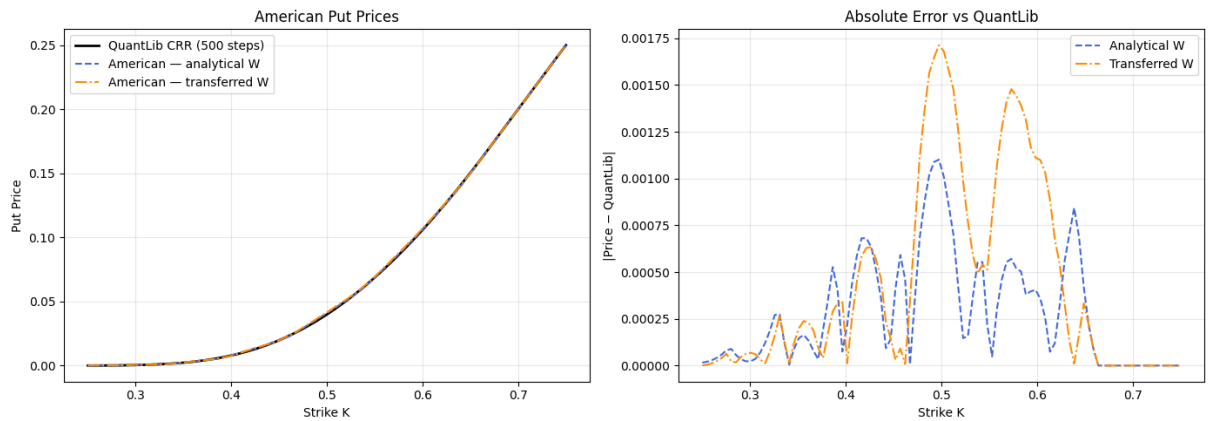


Рисунок 3.1 – Сравнение цен американского пут-опциона и ошибок относительно QuantLib CRR

Таблица 3.1 – Ошибки BTNetAmerican относительно QuantLib CRR с 500 шагами

Вариант инициализации	MAE	RMSE	$\max \text{err} $
Аналитический W	$2.84 \cdot 10^{-4}$	$4.06 \cdot 10^{-4}$	$1.10 \cdot 10^{-3}$
Перенесенный W	$4.38 \cdot 10^{-4}$	$6.81 \cdot 10^{-4}$	$1.71 \cdot 10^{-3}$

При аналитической инициализации получены ошибки $\text{MAE} = 2.84\text{e-}4$, $\text{RMSE} = 4.06\text{e-}4$ и $\max|\text{err}| = 1.10\text{e-}3$. Эти значения малы относительно масштаба цен в рассматриваемом эксперименте. Средняя абсолютная ошибка находится на уровне десятых долей базисного пункта от цены базового актива, а максимальное отклонение остается порядка 10^{-3} . Следовательно, даже дерево глубины $n = 9$ сохраняет приемлемую точность при сравнении с существенно более глубоким эталоном.

Вариант с переносом весов показывает худшие значения: $\text{MAE} = 4.38\text{e-}4$, $\text{RMSE} = 6.81\text{e-}4$ и $\max|\text{err}| = 1.71\text{e-}3$. Ошибки остаются небольшими в абсолютном выражении, однако возрастают по всем трем метрикам. Относительно аналитической инициализации MAE увеличивается примерно в 1.54 раза, RMSE - примерно в 1.68 раза, а максимальное отклонение - примерно в 1.55 раза. Это указывает на систематическое ухудшение, а не на случайный выброс одной точки сетки.

Причина такого эффекта связана с различием европейской и американской задач. Европейская модель обучается на цене, в которой отсутствует право досрочного исполнения. Американская модель на каждом временном слое должна сравнивать продолженную стоимость с внутренней стоимостью. Если фильтр W после обучения на европейских данных отклоняется от риск-нейтральных CRR-весов, то продолженная стоимость в узлах дерева меняется, а ошибка накапливается при обратной индукции.

С точки зрения риск-менеджмента это важный вывод. В задачах оценки чувствительностей недостаточно получить визуально близкую цену: необходимо сохранить финансовую интерпретацию весов и устойчивость локальных производных. Аналитическая инициализация лучше подходит для этой цели, поскольку она напрямую задает риск-нейтральные вероятности и дисконтирование. Перенос весов может быть полезен как исследовательский эксперимент, но не выглядит более надежным способом инициализации американской модели.

Таким образом, верификация прайсинга подтверждает две гипотезы работы. Во-первых, BTNetAmerican с аналитической инициализацией корректно воспроизводит биномиальный механизм оценки американского пут-опциона. Во-вторых, обучение европейской модели не переносится на американскую задачу автоматически: наличие раннего исполнения меняет структуру оптимального решения и требует сохранения CRR-смысла весов.

Различие между BTNet глубины $n = 9$ и QuantLib CRR с 500 шагами следует рассматривать как различие между двумя уровнями дискретизации. QuantLib используется как более точное численное приближение, а BTNet показывает, насколько компактная дифференцируемая архитектура способна воспроизвести цену при малом числе временных слоев. Полученные ошибки показывают, что компактность не приводит к существенной потере качества в выбранной области параметров.

3.2. Греческие символы через автоматическое дифференцирование

Помимо справедливой цены опциона в практических задачах риск-менеджмента важны греческие символы, то есть чувствительности цены к рыночным параметрам. Они позволяют оценивать, как изменится стоимость позиции при малом

изменении цены базового актива, волатильности или времени до экспирации. В настоящей работе рассматриваются четыре грека: Delta, Gamma, Vega и Theta.

Delta определяется как первая производная цены опциона по цене базового актива: $\Delta = dV / dS$. Для пут-опциона Delta обычно принимает отрицательные значения, поскольку рост цены базового актива уменьшает стоимость права продать актив по фиксированной цене. Gamma равна второй производной цены по цене базового актива: $\Gamma = d^2V / dS^2$. Она характеризует выпуклость цены опциона и показывает, насколько быстро меняется Delta.

Vega показывает чувствительность цены опциона к изменению волатильности: $Vega = dV / d\sigma$. Для стандартного опциона рост волатильности обычно увеличивает стоимость, поскольку расширяет диапазон возможных будущих значений базового актива. Theta характеризует изменение цены при течении времени. В работе используется соглашение $\Theta = -dV / dT$, где T обозначает время до экспирации; при таком знаке отрицательная Theta соответствует потере временной стоимости при приближении даты исполнения.

Для вычисления греков в BTNet используется не классическая процедура малых сдвигов с повторной переоценкой, а автоматическое дифференцирование PyTorch. Основная идея состоит в том, чтобы записать CRR/BTNet-проход как функциональное выражение, в котором рыночные параметры S_0 , σ , r и T являются дифференцируемыми тензорами. После вычисления цены опциона PyTorch строит вычислительный граф и позволяет получить производные с помощью `torch.autograd.grad` [8].

Такой функциональный подход отличается от простого вызова уже созданного класса модели. В обычной реализации BTNet веса сети вычисляются в конструкторе из заданных числовых параметров и затем хранятся как параметры слоя. Для вычисления производных по S_0 , σ или T необходимо, чтобы зависимость весов, терминальных выплат и значений дерева от этих параметров оставалась внутри вычислительного графа. Поэтому в модуле `greeks.py` реализованы отдельные функции дифференцируемого CRR-прохода.

Для европейского пут-опциона функциональный проход сначала строит терминальные цены базового актива и выплаты $\max(K - S_T, 0)$, а затем последовательно применяет дисконтированные риск-нейтральные веса CRR. После этого Delta

и Gamma вычисляются как первая и вторая производные по S_0 , Vega - как производная по σ , а Theta - как производная по времени до экспирации с обратным знаком. Полученные значения сравниваются с аналитическими формулами греков в модели Блэка-Шоулса.

Сравнение европейских греков с формулами Блэка-Шоулса выполняет роль контрольного теста. Если функциональный CRR/BNNet-проход построен корректно, то при достаточно большой глубине дерева его цена и чувствительности должны приближаться к аналитическим значениям. В настоящей работе дерево имеет конечную глубину $n = 9$, поэтому небольшие расхождения ожидаемы и интерпретируются как следствие дискретизации, а не как ошибка автоматического дифференцирования.

Для американского пут-опциона ситуация сложнее, поскольку замкнутых аналитических формул для греков в общем случае нет. В функциональном проходе для американского опциона после вычисления продолженной стоимости на каждом узле дополнительно рассчитывается внутренняя стоимость немедленного исполнения. Затем применяется операция $\max$, соответствующая MaxoutLayer в архитектуре BNNetAmerican. Автоматическое дифференцирование проходит через ту ветвь, которая определяет максимум в данном узле.

Особенность американских греков состоит в наличии границы раннего исполнения. В области исполнения цена опциона совпадает с внутренней стоимостью $K - S$, поэтому Delta близка к -1. В области продолжения цена определяется дисконтированным ожиданием будущих выплат, и чувствительности имеют другой характер. На границе между этими областями функция цены имеет излом, что делает производные менее гладкими, чем в европейском случае.

Поскольку для американского пут-опциона нет простой аналитической формулы греков, верификация выполняется с помощью центральных конечных разностей на том же функциональном CRR-проходе. Для Delta цена пересчитывается при $S_0 + h$ и $S_0 - h$, для Gamma используется вторая центральная разность, для Vega изменяется σ , а для Theta - время до экспирации T . Такое сравнение не является независимой рыночной моделью, но оно проверяет согласованность autograd-градиентов с численным дифференцированием той же функции цены.

На рисунке 3.2 показано сравнение греческих символов для аналитического и перенесенного фильтра W. В качестве дополнительного ориентира на графиках приведены европейские греки Блэка-Шоулса; они не являются эталоном для американского опциона, но помогают увидеть отличие кусочно-линейной BTNet-аппроксимации от гладкой аналитической кривой.

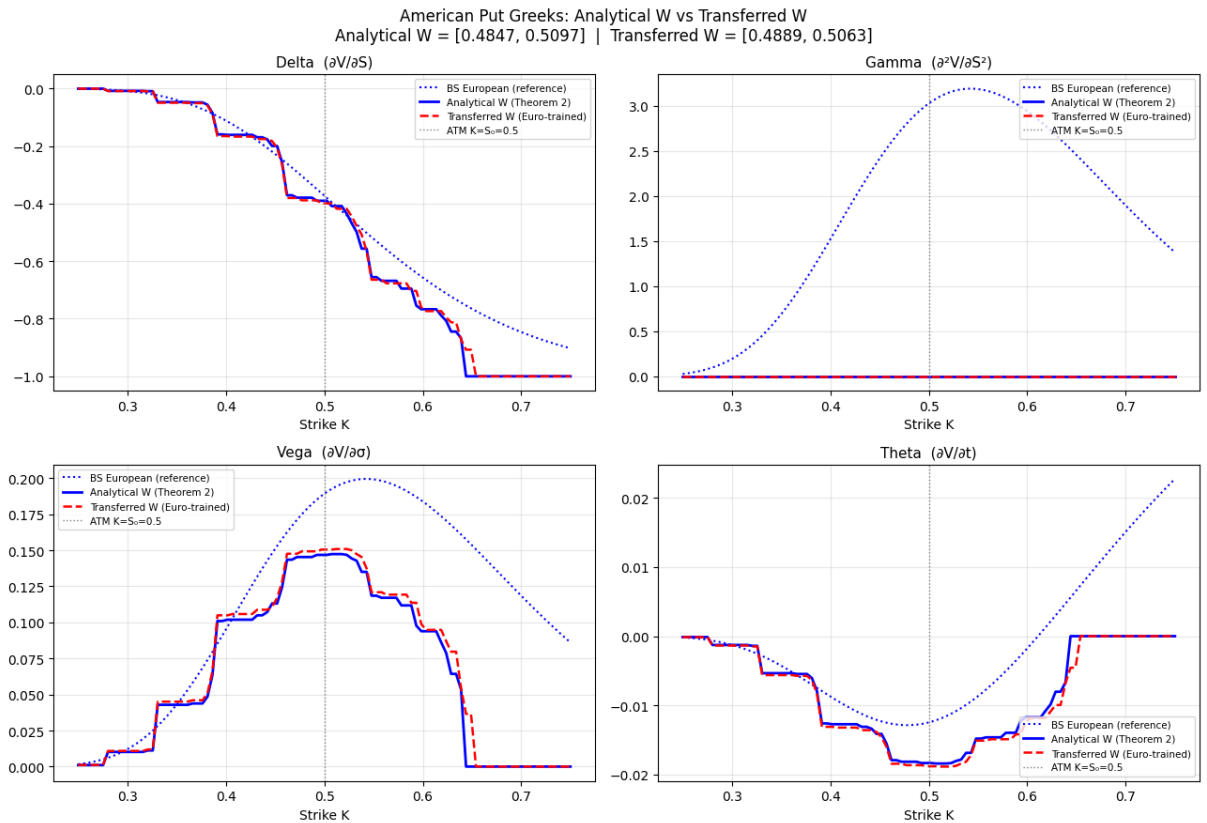


Рисунок 3.2 – Сравнение греческих символов при аналитическом и перенесенном фильтре W

Абсолютные различия между греками аналитической и перенесенной моделей показаны на рисунке 3.3. Наибольшие отклонения наблюдаются для Delta и Vega в области около денег и рядом с участками изменения активной ветви maxout.

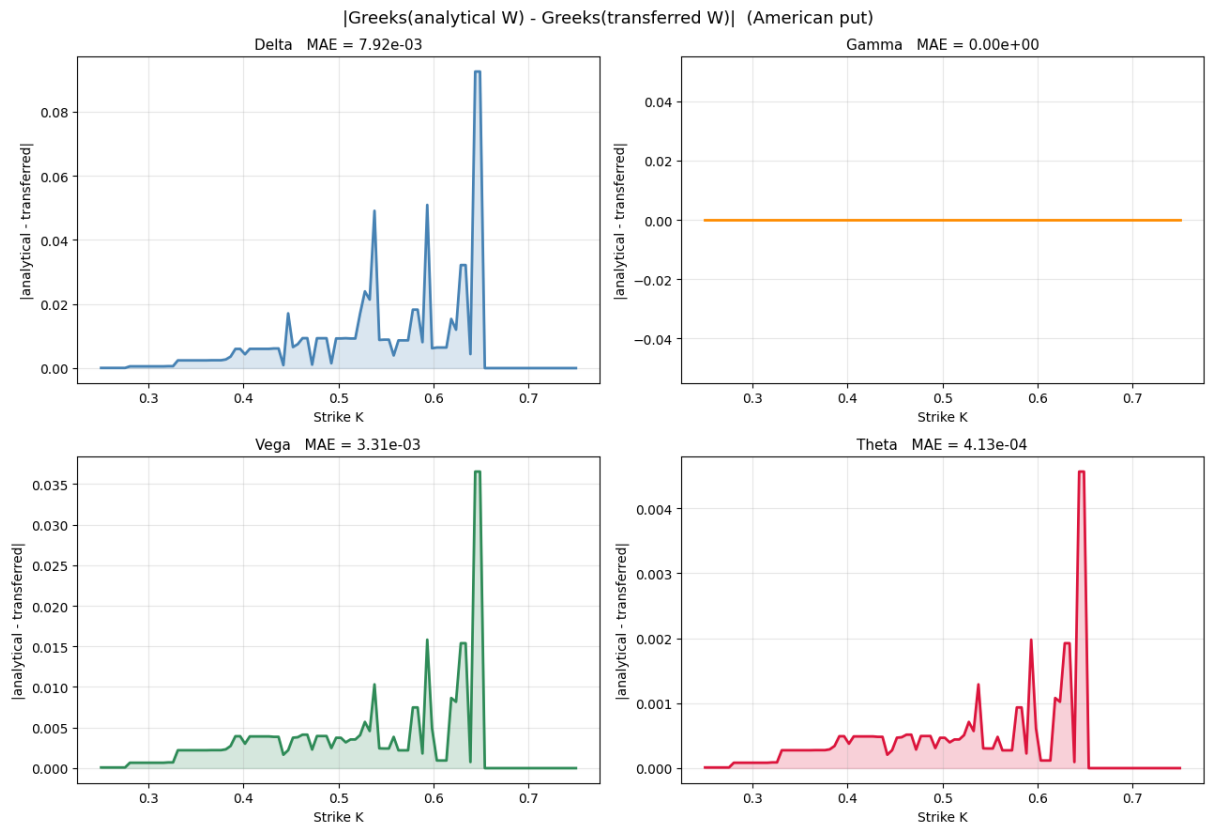


Рисунок 3.3 – Абсолютные различия греческих символов для аналитического и перенесенного фильтра W

Отдельный важный результат связан с поведением Gamma. Архитектуры BTNetEuropean и BTNetAmerican используют ReLU и maxout, то есть кусочно-линейные операции. Следовательно, цена опциона как функция S_0 в такой модели также является кусочно-линейной. Внутри каждого линейного участка первая производная постоянна, а вторая производная равна нулю. Поэтому Gamma, вычисленная средствами autograd, оказывается равной нулю почти всюду.

Нулевая Gamma не является программной ошибкой. Это математическое следствие выбранной архитектуры. В точках излома, где меняется активная ветвь ReLU или maxout, классическая вторая производная не определена. PyTorch в таких точках работает с субградиентами и не восстанавливает сингулярный вклад, который в непрерывной финансовой модели соответствует скачку Delta. Поэтому эффект $\text{Gamma} = 0$ следует рассматривать как ограничение кусочно-линейной BTNet-архитектуры при анализе выпуклости цены опциона.

Для практического применения это означает, что Delta, Vega и Theta могут быть получены напрямую через автоматическое дифференцирование и использоваться для первичного анализа рыночного риска. Однако стратегии, требующие

точной оценки Gamma, например дельта-гамма-хеджирование, нуждаются в более гладкой архитектуре. Одним из возможных направлений развития является замена ReLU и maxout на гладкие приближения, например Softplus, что потенциально позволит получить ненулевую и более устойчивую Gamma.

Таблица 3.2 – Проверка американских греков центральными конечными разностями

Грек	MAE	max diff
Delta	$7.92 \cdot 10^{-3}$	$9.25 \cdot 10^{-2}$
Gamma	0	0
Vega	$3.31 \cdot 10^{-3}$	$3.65 \cdot 10^{-2}$
Theta	$4.13 \cdot 10^{-4}$	$4.57 \cdot 10^{-3}$

Полученные значения показывают, что автоматическое дифференцирование согласуется с центральными конечными разностями для основных чувствительностей. Наиболее заметное расхождение наблюдается для Delta: это ожидаемо, поскольку Delta американского пут-опциона меняется скачкообразно вблизи границы раннего исполнения. Если малое изменение S_0 переводит отдельный узел дерева из области продолжения в область исполнения, конечная разность и autograd могут выбрать разные локальные ветви функции max.

Для Vega и Theta ошибки меньше, потому что изменение волатильности или времени до экспирации обычно влияет на все дерево более плавно. При этом и здесь сохраняется зависимость от активных ветвей maxout: если при изменении параметра меняется решение об исполнении в некотором узле, чувствительность становится менее гладкой. Поэтому американские греки в дискретной модели следует интерпретировать как локальные чувствительности конкретной аппроксимации, а не как гладкие производные непрерывной задачи.

Отдельного внимания требует Gamma. В классической модели Блэка-Шоулса цена европейского опциона является гладкой функцией S , поэтому Gamma положительна и отражает выпуклость стоимости [2; 3]. В реализованной BTNet-архитектуре ситуация другая: ReLU и maxout задают кусочно-линейную функцию. Внутри каждого линейного участка Delta постоянна, следовательно, вторая производная равна нулю. В точках излома классическая Gamma не определена, а PyTorch возвращает производную выбранной ветви вычислительного графа [8].

Таким образом, нулевая Gamma в таблице 3.2 является не ошибкой реализации, а диагностикой архитектурного ограничения. Для задач, где достаточно Delta, Vega и Theta, такой подход остается полезным благодаря простой дифференцируемости и прямой связи с CRR. Для задач дельта-гамма-хеджирования требуется дальнейшее развитие модели: например, замена ReLU и maxout на гладкие приближения или использование сглаженной процедуры раннего исполнения.

3.3. Эксперимент переноса весов из европейской модели в американскую

Эксперимент переноса весов из BTNetEuropean в BTNetAmerican был включен в работу как проверка практической гипотезы о переиспользовании обученной нейросетевой структуры. Эта гипотеза выглядит естественной: обе архитектуры строятся на одной и той же биномиальной геометрии CRR, используют терминальный слой выплат и сверточный фильтр W , а отличие американского случая состоит в добавлении операции раннего исполнения. Поэтому можно предположить, что обучение европейской модели на гладком аналитическом эталоне даст более точный фильтр для последующего американского прайсинга.

Однако с финансовой точки зрения эта гипотеза не является очевидной. Европейский пут-опцион оценивается только по терминальной выплате, тогда как американский пут содержит решение о досрочном исполнении в каждом узле дерева. Следовательно, фильтр W в американской модели участвует не просто в усреднении будущих значений, а в сравнении продолженной стоимости с внутренней стоимостью. Даже малое изменение W способно изменить границу исполнения, а значит и всю траекторию обратной индукции [1; 4].

Протокол эксперимента состоит из четырех шагов. Сначала создается модель BTNetEuropean с параметрами $S_0 = 0.5$, $T = 1$, $r = 0.05$, $\sigma = 0.25$ и $n = 9$. Затем она обучается методом Adam на ценах европейского пут-опциона, рассчитанных по формуле Блэка-Шоулса-Мертона [2; 3; 9]. После обучения из модели извлекаются параметры начального слоя и сверточный фильтр W . Наконец, эти параметры копируются в модель BTNetAmerican, после чего все веса замораживаются и дополнительное обучение на американских ценах не проводится.

Динамика обучения европейской модели, веса которой затем переносятся в американскую архитектуру, приведена на рисунке 3.4. Значение функции потерь

быстро уменьшается в первые эпохи, после чего выходит на устойчивый низкий уровень.

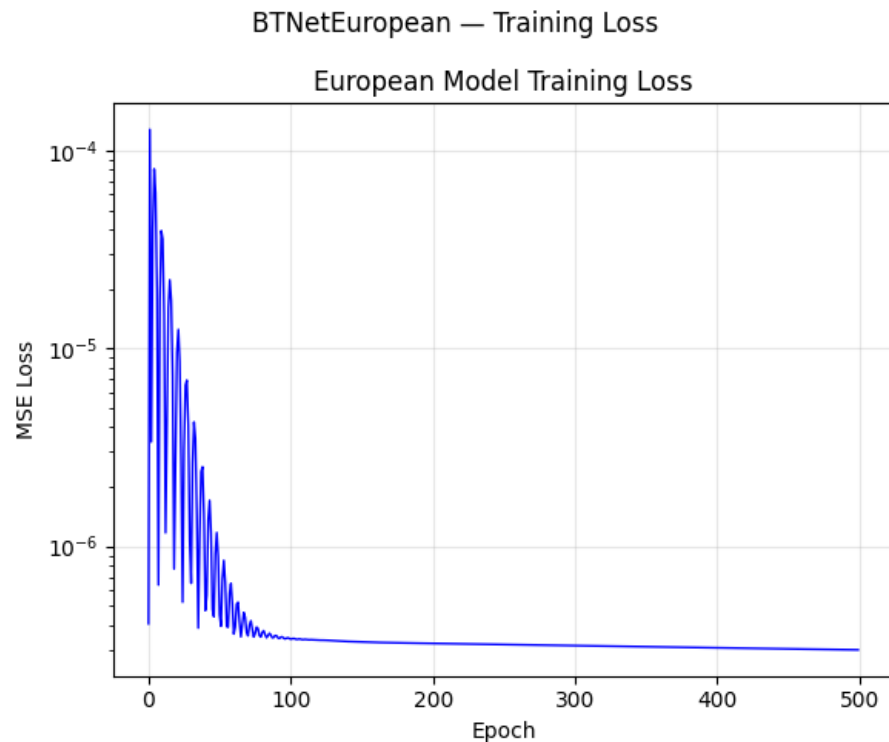


Рисунок 3.4 – Динамика функции потерь при обучении BTNetEuropean

Такое замораживание принципиально важно для чистоты эксперимента. Если бы после переноса выполнялось дополнительное обучение на американских ценах, то было бы невозможно понять, что именно повлияло на результат: исходный перенос или последующая подгонка под новую целевую функцию. В выбранной постановке проверяется именно качество перенесенной европейской структуры как готовой инициализации для американской задачи.

В таблице 3.3 приведено сравнение аналитического и перенесенного фильтра W . Аналитические веса соответствуют дисконтированным риск-нейтральным коэффициентам CRR. Перенесенные веса получены после обучения европейской модели и затем скопированы во все тахout-слои американской архитектуры.

Таблица 3.3 – Сравнение аналитического и перенесенного фильтра W

Вариант фильтра W	w_0	w_1	Сумма весов	Интерпрета- ция
Аналитический CRR	0.4847	0.5097	0.9944	Дисконтиро- ванное риск-

				нейтральное ожидание
Перенесенный из BTNetEuropean	0.4889	0.5063	0.9952	Фильтр, под- строенный под европей- скую цену
Разность пере- несенный - ана- литический	+0.0042	-0.0034	+0.0008	Смещение продолженной стоимости в узлах дерева

Численно отклонение кажется небольшим: каждая компонента меняется примерно на четыре тысячных. Тем не менее для американского опциона такое изменение оказывается существенным. При аналитической инициализации ошибка относительно QuantLib CRR с 500 шагами составляет $MAE = 2.84e-4$, $RMSE = 4.06e-4$ и $\max|err| = 1.10e-3$. При перенесенном фильтре ошибки возрастают до $MAE = 4.38e-4$, $RMSE = 6.81e-4$ и $\max|err| = 1.71e-3$. Ухудшение наблюдается по всем метрикам, поэтому его нельзя объяснить одной случайной точкой тестовой сетки.

Основная причина ухудшения заключается в том, что европейское обучение пытается компенсировать конечную глубину дерева. Модель BTNetEuropean с $n = 9$ приближает непрерывную цену Блэка-Шоулса, и оптимизатор может немного сдвинуть W от строгих CRR-коэффициентов, чтобы уменьшить среднюю ошибку на европейской задаче. Для европейского опциона такой сдвиг может выглядеть полезным. Для американского опциона он нарушает экономический смысл продолженной стоимости, потому что $\maxout$ -слой сравнивает уже смещенное продолжение с внутренней стоимостью исполнения.

Накопление ошибки происходит рекурсивно. На первом шаге обратной индукции изменяется стоимость продолжения в узлах перед экспирацией. Затем эти измененные значения становятся входом для следующего слоя, и смещение передается назад к текущему моменту. Если в некотором узле продолженная стоимость оказывается близкой к внутренней, даже небольшой сдвиг W может изменить активную ветвь $\maxout$. Поэтому американская цена чувствительна не только к величине фильтра, но и к тому, как этот фильтр влияет на решение об исполнении.

Отдельно проявляется влияние на греческие символы. В эксперименте с фиксированным W сравнивались чувствительности аналитической и перенесенной американской модели. Для Delta среднее расхождение составляет примерно $7.9e-3$, для Vega - около $3.3e-3$. Это показывает, что производные цены могут реагировать на изменение весов сильнее, чем сама цена. Такая ситуация естественна: чувствительности измеряют локальный наклон функции стоимости, а локальный наклон особенно зависит от выбранной ветви `maxout`.

Для Vega и Theta возникает еще одна методологическая проблема. В аналитической модели фильтр W зависит от σ , r и T через параметры CRR, поэтому при автоматическом дифференцировании градиент проходит не только через выплаты и геометрию дерева, но и через риск-нейтральные вероятности. В перенесенной модели W фиксирован, поэтому этот канал чувствительности отсутствует. Следовательно, перенесенная модель становится менее корректной именно как инструмент риск-менеджмента, даже если ее цена визуально близка к эталону.

Полученный результат согласуется с теоретической интерпретацией BTNet [5]. Эквивалентность между биномиальным деревом и нейросетью выполняется при специальных весах, имеющих финансовый смысл. Она не означает, что любые веса, обученные на смежной задаче, будут сохранять этот смысл. Напротив, эксперимент показывает границу применимости машинного обучения в такой архитектуре: обучение может улучшать аппроксимацию одной целевой функции, но одновременно ухудшать структурную согласованность с другой задачей.

Практический вывод состоит в том, что для американского пут-опциона предпочтительна аналитическая инициализация BTNetAmerican. Она сохраняет связь с риск-нейтральной мерой, обеспечивает меньшую ошибку прайсинга и дает более надежные чувствительности. Перенос весов из европейской модели в американскую полезен как диагностический эксперимент, но не должен использоваться как основной способ построения модели для задач оценки риска.

Заключение

В ходе выпускной квалификационной работы была достигнута поставленная цель: реализована и верифицирована дифференцируемая нейросетевая архитектура BTNet для оценки американских пут-опционов и вычисления их греческих символов. Итогом работы стал программный комплекс `bttn_bs`, в котором биномиальная модель CRR представлена в форме нейросетевых слоев PyTorch, а результаты прайсинга сопоставлены с эталонными расчетами QuantLib.

Теоретическая часть работы показала, что американский пут-опцион принципиально отличается от европейского наличием права досрочного исполнения. Это превращает задачу оценки в задачу оптимальной остановки, для которой в общем случае нет простой аналитической формулы. Поэтому особое значение получают численные методы, прежде всего биномиальная модель Кокса-Росса-Рубинштейна. Архитектура BTNet использует тот же механизм обратной индукции, но записывает его как последовательность дифференцируемых операций.

В программной части были реализованы базовые слои `DenseLayer`, `ConvLayer` и `MaxoutLayer`, а также модели `BTNetEuropean` и `BTNetAmerican`. `DenseLayer` формирует терминальные выплаты пут-опциона, `ConvLayer` реализует дисконтированное риск-нейтральное ожидание, а `MaxoutLayer` добавляет сравнение продолженной стоимости с внутренней стоимостью исполнения. Такая структура сохраняет финансовую интерпретируемость модели: веса не являются произвольными параметрами, а соответствуют элементам CRR-дерева.

Верификация прайсинга подтвердила корректность реализации. Для американского пут-опциона аналитически инициализированная `BTNetAmerican` сравнивалась с QuantLib CRR с 500 шагами на сетке страйков от 0.25 до 0.75 при параметрах $S_0 = 0.5$, $T = 1$, $r = 0.05$, $\sigma = 0.25$ и $n = 9$. Получены ошибки $MAE = 2.84e-4$, $RMSE = 4.06e-4$ и $\max|err| = 1.10e-3$. Эти значения показывают, что даже компактное дерево глубины $n = 9$ дает близкое приближение к более детальному эталонному расчету.

Отдельным результатом стало исследование переноса весов из европейской модели в американскую. Первоначальная гипотеза состояла в том, что обучение `BTNetEuropean` на ценах Блэка-Шоулса может дать полезный фильтр W для американской архитектуры. Численный эксперимент показал обратное: при

перенесенном фильтре ошибки возрастают до $MAE = 4.38e-4$, $RMSE = 6.81e-4$ и $\max|err| = 1.71e-3$. Это означает, что веса, подстроенные под европейскую задачу, смещаются относительно риск-нейтральных CRR-коэффициентов и ухудшают расчет раннего исполнения.

В работе также были вычислены греческие символы Delta, Gamma, Vega и Theta. Для этого использовался функциональный CRR/BTNet-проход, в котором рыночные параметры задаются как дифференцируемые тензоры. Такой подход позволяет применять `torch.autograd.grad` и получать чувствительности без ручного вывода отдельных формул для каждой модели. Для европейского опциона результаты сопоставлялись с аналитическими формулами Блэка-Шоулса, а для американского опциона проверялись центральными конечными разностями.

Наиболее важное ограничение связано с поведением Gamma. Поскольку реализованная BTNet-архитектура использует ReLU и `maxout`, цена опциона как функция входных параметров является кусочно-линейной. Внутри линейных участков вторая производная равна нулю, а в точках излома классическая производная не определена. Поэтому Gamma, вычисленная средствами автоматического дифференцирования, оказывается равной нулю почти всюду. Это не является ошибкой программы, но ограничивает применение такой архитектуры в задачах, где необходима точная оценка выпуклости.

Практическая значимость работы состоит в том, что разработанный пакет `bttn_bs` можно использовать как основу для воспроизводимых экспериментов по дифференцируемому прайсингу. Он объединяет интерпретируемую финансовую структуру CRR, реализацию на PyTorch, верификацию через QuantLib и вычисление чувствительностей. Наиболее надежной для американского пут-опциона оказалась аналитическая инициализация весов, поскольку она сохраняет экономический смысл риск-нейтрального ожидания.

Ограничения исследования определяются выбранной постановкой экспериментов. Все основные расчеты выполнены при фиксированных рыночных параметрах и глубине дерева $n = 9$. Кроме того, эталон QuantLib использует гораздо более глубокое дерево, поэтому часть ошибки связана не с архитектурой BTNet, а с различием уровней дискретизации. Наконец, кусочно-линейный характер

ReLU/maout ограничивает качество оценки Gamma и требует осторожной интерпретации результатов при анализе хеджирования.

Перспективы дальнейшего развития включают несколько направлений. Во-первых, можно заменить ReLU и maout на гладкие приближения, например Softplus или GELU, чтобы получить ненулевую и более устойчивую Gamma. Во-вторых, модель может быть расширена на локальную и подразумеваемую волатильность, где параметры дерева зависят от состояния рынка. В-третьих, представляет интерес применение BTNet к многомерным активам, барьерным и другим экзотическим опционам. Наконец, отдельным направлением является калибровка модели на реальных рыночных данных.

Таким образом, работа показывает, что BTNet можно рассматривать не как черный ящик, а как интерпретируемую дифференцируемую форму биномиального дерева. Для американского пут-опциона такая форма особенно полезна: она сохраняет логику раннего исполнения, позволяет проверять цену относительно численного эталона и открывает путь к автоматическому вычислению чувствительностей. При этом эксперимент с переносом весов показывает, что финансово осмысленная аналитическая инициализация остается более надежной, чем обучение на смежной европейской задаче.

Список используемых источников

1. Hull J.C. Options, Futures, and Other Derivatives. — 11th ed. — Pearson, 2021.
2. Black F., Scholes M. The pricing of options and corporate liabilities // Journal of Political Economy. — 1973. — Vol. 81, No. 3. — P. 637–654.
3. Merton R.C. Theory of rational option pricing // The Bell Journal of Economics and Management Science. — 1973. — Vol. 4, No. 1. — P. 141–183.
4. Cox J.C., Ross S.A., Rubinstein M. Option pricing: A simplified approach // Journal of Financial Economics. — 1979. — Vol. 7, No. 3. — P. 229–263.
5. Шорохов С. Г. Оценка финансовых деривативов нейронной сетью на основе биномиального дерева // International Journal of Open Information Technologies. — 2024. — Т. 12, № 5. — С. 108–114.
6. Liu S., Oosterlee C.W., Bohte S.M. Pricing Options and Computing Implied Volatilities using Neural Networks // Risks. — 2019. — Vol. 7, No. 1. — P. 16. — DOI: 10.3390/risks7010016.
7. Longstaff F.A., Schwartz E.S. Valuing American Options by Simulation: A Simple Least-Squares Approach // The Review of Financial Studies. — 2001. — Vol. 14, No. 1. — P. 113–147.
8. Paszke A. et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library // Advances in Neural Information Processing Systems 32. — Curran Associates, Inc., 2019. — P. 8024–8035.
9. Kingma D.P., Ba J. Adam: A Method for Stochastic Optimization // Proc. of the 3rd International Conference on Learning Representations (ICLR). — 2015.
10. QuantLib Python Documentation. — URL: <https://quantlib-python-docs.readthedocs.io>.
11. Virtanen P. et al. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python // Nature Methods. — 2020. — Vol. 17. — P. 261–272. — DOI: 10.1038/s41592-019-0686-2.
12. Hunter J.D. Matplotlib: A 2D Graphics Environment // Computing in Science & Engineering. — 2007. — Vol. 9, No. 3. — P. 90–95.

Приложение А. Исходный код пакета btnn_bs

А.1. Базовые слои (layers.py)

Модуль layers.py содержит три базовых слоя, из которых собираются архитектуры BTNet. ConvLayer — одномерная свертка с фиксированным фильтром $[W_0, W_1]$, реализующая дискретное дисконтирование CRR на одном шаге назад. DenseLayer — полносвязный слой с необязательной функцией активации, используемый для инициализации терминальных выплат по листьям дерева. MaxoutLayer — комбинирует ConvLayer (продолженная стоимость) и линейный слой (стоимость раннего исполнения), возвращая их поэлементный максимум — ключевую операцию обратной индукции для американского опциона.

```
class ConvLayer(nn.Module):
    def __init__(self, filter_weight=None):
        super().__init__()
        self._conv_1d = nn.Conv1d(
            in_channels=1, out_channels=1, kernel_size=2, bias=False,
        )
        if filter_weight is not None:
            with torch.no_grad():
                self._conv_1d.weight.data = torch.as_tensor(
                    filter_weight.reshape(1, 1, 2), dtype=torch.float32,
                )
```

```
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = x.unsqueeze(1)
        x = self._conv_1d(x)
        return x.squeeze(1)
```

```
class DenseLayer(nn.Module):
    def __init__(self, input_dim, output_dim, W=None, bias=None, transformation=None):
        super().__init__()
```

```

self._linear = nn.Linear(input_dim, output_dim, bias=True)
if W is not None:
    with torch.no_grad():
        W = torch.as_tensor(W, dtype=torch.float32)
        if W.dim() == 1:
            W = W.unsqueeze(0)
        self._linear.weight.data = W.T if W.shape[0] == input_dim else W
if bias is not None:
    with torch.no_grad():
        self._linear.bias.data = torch.as_tensor(bias, dtype=torch.float32).flat-
ten()

self._transformation = transformation if transformation is not None else
(lambda t: t)

def forward(self, x: torch.Tensor) -> torch.Tensor:
    return self._transformation(self._linear(x))

class MaxoutLayer(nn.Module):
    def __init__(self, input_dim, output_dim, filter_weight=None, W_linear=None,
bias=None):
        super().__init__()
        self._conv_1d = nn.Conv1d(in_channels=1, out_channels=1, kernel_size=2,
bias=False)
        self._linear = nn.Linear(in_features=1, out_features=output_dim, bias=True)
        if filter_weight is not None:
            with torch.no_grad():
                self._conv_1d.weight.data = torch.as_tensor(
                    filter_weight.reshape(1, 1, 2), dtype=torch.float32,
                )
        if W_linear is not None:
            with torch.no_grad():

```

```

        W_linear = torch.as_tensor(W_linear, dtype=torch.float32)
        self._linear.weight.data = W_linear.unsqueeze(0).T
        if bias is not None:
            with torch.no_grad():
                self._linear.bias.data = torch.as_tensor(bias, dtype=torch.float32).flatten()

def forward(self, v_prev: torch.Tensor, k: torch.Tensor) -> torch.Tensor:
    v_conv = self._conv_1d(v_prev.unsqueeze(1)).squeeze(1)
    v_linear = self._linear(k)
    return torch.max(v_conv, v_linear)

```

A.2. Архитектура BTNetEuropean (model_european.py)

BTNetEuropean реализует оценку европейского пут-опциона. В конструкторе аналитически вычисляются параметры CRR (u , d , дисконтированные веса W_0 , W_1) и инициализируются веса DenseLayer (начальные выплаты) и ConvLayer (сверточный фильтр). Метод forward выполняет n последовательных применений ConvLayer, что эквивалентно прямой индукции по биномиальному дереву.

```

class BTNetEuropean(nn.Module):
    def __init__(self, n_dim, S0, sig, T, t0, r=None):
        super().__init__()
        self._r = r if r is not None else 0.05
        dt = (T - t0) / n_dim
        sqrt_dt = np.sqrt(dt)
        u = np.exp(sig * sqrt_dt)
        d = np.exp(-sig * sqrt_dt)

        w = np.array([
            (np.exp(-self._r * dt) * u - 1) / (u - d),
            (1 - np.exp(-self._r * dt) * d) / (u - d),
        ], dtype=np.float32)

```

```

w_init = np.ones((1, n_dim + 1), dtype=np.float32)
b_init = np.array(
    [-S0 * np.exp(sig * sqrt_dt * (2*j - n_dim)) for j in range(n_dim + 1)],
    dtype=np.float32,
)

self._initial_layer = DenseLayer(
    input_dim=1, output_dim=n_dim + 1,
    W=w_init, bias=b_init, transformation=torch.relu,
)

self._conv_layer = ConvLayer(filter_weight=w)
self._n_dim = n_dim

def forward(self, k: torch.Tensor) -> torch.Tensor:
    if k.dim() == 1:
        k = k.unsqueeze(1)
    k = k.float()
    x = self._initial_layer(k)
    for _ in range(self._n_dim):
        x = self._conv_layer(x)
    return x.squeeze(1) if x.size(1) == 1 else x.mean(dim=1, keepdim=True)

```

А.3. Архитектура BTNetAmerican (model_american.py)

BTNetAmerican реализует оценку американского пут-опциона. Вместо ConvLayer на каждом шаге используется MaxoutLayer: он вычисляет как продолженную стоимость (через свертку), так и стоимость немедленного исполнения (через линейный слой, кодирующий цены узлов дерева на текущем шаге), и возвращает их поэлементный максимум. Таким образом, forward BTNetAmerican является прямым аналогом обратной индукции CRR с условием раннего исполнения.

```

class BTNetAmerican(nn.Module):
    def __init__(self, n_dim, S0, sig, T, t0, r=None):
        super().__init__()

```

```

self._r = r if r is not None else 0.05
dt      = (T - t0) / n_dim
sqrt_dt = np.sqrt(dt)
u       = np.exp(sig * sqrt_dt)
d       = np.exp(-sig * sqrt_dt)

self._W = np.array([
    (np.exp(-self._r * dt) * u - 1) / (u - d),
    (1 - np.exp(-self._r * dt) * d) / (u - d),
], dtype=np.float32)

w_init = np.ones((1, n_dim + 1), dtype=np.float32)
b_init = np.array(
    [-S0 * np.exp(sig * sqrt_dt * (2*j - n_dim)) for j in range(n_dim + 1)],
    dtype=np.float32,
)

self._initial_layer = DenseLayer(
    input_dim=1, output_dim=n_dim + 1,
    W=w_init, bias=b_init, transformation=torch.relu,
)

self._maxout_layers = nn.ModuleList()
for i in range(n_dim):
    out_dim = n_dim - i
    w_linear = np.ones((1, out_dim), dtype=np.float32)
    b_linear = np.array(
        [-S0 * np.exp(sig * sqrt_dt * (2*j - (n_dim - 1 - i)))
         for j in range(out_dim)],
        dtype=np.float32,
    )
    self._maxout_layers.append(
        MaxoutLayer(input_dim=n_dim + 1 - i, output_dim=out_dim,

```

```

        filter_weight=self._W, W_linear=w_linear, bias=b_linear)
    )

def forward(self, k: torch.Tensor) -> torch.Tensor:
    if k.dim() == 1:
        k = k.unsqueeze(1)
    k = k.float()
    v = self._initial_layer(k)
    for maxout_layer in self._maxout_layers:
        v = maxout_layer(v, k)
    return v.view(k.size(0), -1)[: , :1]

```

A.4. Функция обучения `train_BTNet (training.py)`

Функция `train_BTNet` реализует стандартный цикл обучения для любой архитектуры `BTNet`. В качестве оптимизатора используется `Adam` с настраиваемым шагом обучения `lr`; функция потерь — среднеквадратическая ошибка (`MSELoss`). Обновляются только обучаемые параметры (`requires_grad=True`), что позволяет замораживать аналитически инициализированные веса при необходимости. Функция возвращает историю значений функции потерь по эпохам.

```

def train_BTNet(model, K_train, prices_train, epochs=200, lr=0.01,
log_every=50):
    trainable = [p for p in model.parameters() if p.requires_grad]
    optimizer = torch.optim.Adam(trainable, lr=lr)
    loss_fn = nn.MSELoss()

    K_train = torch.from_numpy(K_train).float().unsqueeze(1)
    prices_train = torch.from_numpy(prices_train).float().unsqueeze(1)

    loss_history = []

    for epoch in range(epochs):
        model.train()

```

```

optimizer.zero_grad()

predictions = model(K_train)
if predictions.shape != prices_train.shape:
    predictions = predictions.view_as(prices_train)

loss = loss_fn(predictions, prices_train)
loss.backward()
optimizer.step()

loss_history.append(loss.item())
if log_every and (epoch + 1) % log_every == 0:
    print(f"Epoch [{epoch + 1}/{epochs}], Loss: {loss.item():.6f}")

return loss_history

```

A.5. Вычисление греческих символов через autograd (greeks.py)

Функция `btnet_greeks` вычисляет Delta, Gamma, Vega и Theta европейского пут-опциона через механизм `torch.autograd.grad`. Ключевая идея: вместо использования весов обученной сети вызывается функциональный проход `_crr_european_put`, в котором все рыночные параметры (S_0 , σ , r , T) переданы как дифференцируемые тензоры PyTorch. Это позволяет `autograd propagate` градиенты через всю цепочку операций CRR. Delta вычисляется первой производной по S_0 (с `create_graph=True` для последующего взятия Gamma), Gamma — второй производной. Vega и Theta вычисляются в отдельных проходах с `requires_grad` по σ и T соответственно.

```

def _crr_european_put(K, S0, sigma, r, T, n=9):
    dt = T / n
    sqrt_dt = torch.sqrt(dt)
    u = torch.exp(sigma * sqrt_dt)
    d = torch.exp(-sigma * sqrt_dt)
    W0 = (torch.exp(-r * dt) * u - 1.0) / (u - d)

```

```

W1 = (1.0 - torch.exp(-r * dt) * d) / (u - d)

j = torch.arange(n + 1, dtype=torch.float32)
S_T = S0 * torch.exp(sigma * sqrt_dt * (2.0 * j - n))
V = torch.relu(K - S_T)

for _ in range(n):
    V = W0 * V[:-1] + W1 * V[1:]
return V.squeeze()

def btnet_greeks(K_values, S0, sigma, r, T, n=9):
    K_flat = K_values.flatten()
    deltas, gammas, vegas, thetas = [], [], [], []

    for K_val in K_flat:
        K = torch.tensor(K_val, dtype=torch.float32)

        # Delta и Gamma (производные по S0)
        S0_t = torch.tensor(S0, dtype=torch.float32, requires_grad=True)
        V = _crr_european_put(K, S0_t, torch.tensor(sigma), torch.tensor(r),
torch.tensor(T), n)
        (delta,) = torch.autograd.grad(V, S0_t, create_graph=True)
        deltas.append(delta.item())
        (gamma,) = torch.autograd.grad(delta, S0_t)
        gammas.append(gamma.item())

        # Vega (производная по sigma)
        sigma_t = torch.tensor(sigma, dtype=torch.float32, requires_grad=True)
        V2 = _crr_european_put(K, torch.tensor(S0), sigma_t, torch.tensor(r),
torch.tensor(T), n)
        (vega,) = torch.autograd.grad(V2, sigma_t)

```

```

vegas.append(vega.item())

# Theta = -dV/dT (производная по T, со знаком минус)
T_t = torch.tensor(T, dtype=torch.float32, requires_grad=True)
V3 = _crr_european_put(K, torch.tensor(S0), torch.tensor(sigma), torch.tensor(r), T_t, n)
(dV_dT,) = torch.autograd.grad(V3, T_t)
thetas.append(-dV_dT.item())

return {
    "delta": np.array(deltas), "gamma": np.array(gammas),
    "vega": np.array(vegas), "theta": np.array(thetas),
}

```